ПМ.12 / Прототип мобильной 2D-игры

Задание и методические рекомендации

01 / ПМ.12 · учебная практика

Паспорт практики

1

Название и период

Учебная практика ПМ.12 «Программирование мобильных игр» для ДКИП-401рвмп проходит с 10 по 30 ноября 2026 года. Комплект нужно прикрепить в личном кабинете до 30 ноября 2026 года.

2

Первый шаг. Определите свой вариант

Найдите свою фамилию в списке группы ДКИП-401рвмп и возьмите её порядковый номер. Откройте отдельный PDF «Список предметных областей» и найдите карточку с этим номером. Если вы стоите в строке 7, ваш вариант 07, а имя проекта PM12_Variant07. Запишите номер, название и правило в README и отчёт. Если списка нет, фамилия пропущена или в PDF нет вашего номера, уточните номер у руководителя практики до начала работы.

3

Среда работы

Unity 6.6 (6000.6.2f1), шаблон 2D, язык C#. Для примера не нужны платные ресурсы, внешние изображения или сторонние плагины. Проверка выполняется в Game View; сборка под Android является дополнительным результатом.

4

Общее задание

Создайте работающий прототип мобильной 2D-игры по своей карточке. Игрок управляет объектом экранными кнопками, взаимодействует с предметами и ограничениями, выполняет правило варианта и получает результат «Победа» или «Поражение».

5

Один собственный проект

Работайте в проекте PM12_VariantNN. Сначала создайте общую основу по разобранному коду, затем перенесите на неё правило из своей карточки. «Кристальный маршрут» показывает устройство основы. На сдачу нужен один проект вашей игры.

6

Учебный пример

В инструкции разобрана игра «Кристальный маршрут»: нужно собрать пять кристаллов, сохранить три жизни и войти в зелёный портал. Этот пример показывает технические приёмы. Студент переносит их на свою предметную область, а не ограничивается заменой названий и цветов.

7

Репозиторий проекта

В разделе «Результат и порядок сдачи» показано, как загрузить свой проект, проверить ссылку и оформить ответ в ЛМС.

8

Исходные данные и ресурсы

Исходные данные — условия своей карточки: объекты, числа, типы, порядок, победа и поражение. Отдельного стенда варианта нет: Unity-проект создаётся во время задания. Формы объектов создаются кодом; внешние изображения, готовая сцена, платные ресурсы и плагины для обязательного результата не нужны.

02 / Наблюдаемый результат

Что обязательно должно работать

1

Управление

Объект игрока перемещается вверх, вниз, влево и вправо. В Game View работают клавиши WASD или стрелки. На экране есть четыре крупные кнопки, пригодные для последующей проверки на мобильном устройстве.

2

Игровые взаимодействия

Есть целевые объекты, опасности или ограничения и зона завершения. Их количество задаёт карточка варианта: общее требование пяти предметов не применяется к вариантам с иной целью. Контакт меняет значимое состояние игры; повторный контакт не должен повторно засчитывать уже выполненное действие.

3

Завершение

Условия победы и поражения различаются и проверяются отдельно. После завершения ввод блокируется, появляется понятная панель результата и доступна команда повторного запуска без перезапуска Unity.

4

Обратная связь

На экране видны текущая цель, прогресс, оставшиеся попытки или другой значимый показатель. Неверное действие сопровождается короткой подсказкой. Смысл объектов различим не только по цвету, но и по форме или подписи.

03 / Семь этапов · один проект

Последовательность одного задания

1

1–2. Правила и основа проекта

1. Запишите цель, игровые объекты, индивидуальное правило, победу и поражение. 2. Создайте 2D-проект, сцену Main и папку Scripts. После каждого действия сверяйте имя и видимый результат.

2

3–4. Управление и взаимодействия

3. Подключите PlayerController и проверьте движение в четырёх направлениях. 4. Добавьте собираемые объекты, опасности и зону завершения. Сначала проверяйте один объект каждого типа, затем размножайте его.

3

5. Индивидуальное правило

Реализуйте правило своего варианта: выбор, порядок, доставка, вместимость, комплект или изменение доступности. Это обязательная часть программы; одной замены названий недостаточно.

4

6–7. Завершение и сдача

6. Добавьте победу, поражение, блокировку управления и повторный запуск. 7. Проведите проверки, устраните ошибки, сохраните сцену и запишите фактические результаты проверки в отчёте.

05 / Выполнить по порядку

Проверочные сценарии

1

Проверка 1 · запуск

Нажмите Play и не нажимайте кнопки пять секунд. Ожидается: игрок стоит в начальной точке, начальные показатели верны, игра сама не завершается, Console не показывает красных сообщений.

2

Проверка 2 · управление

По очереди удерживайте вверх, вниз, влево и вправо. Ожидается: движение соответствует кнопке; диагональ не быстрее прямого движения; стены не пропускают игрока за поле.

3

Проверка 3 · правило варианта

Выполните одно правильное и одно неправильное действие, указанное в карточке варианта. Ожидается: показатели меняются строго по правилу, а неверное действие не засчитывается как верное.

4

Проверки 4–5 · конец и повтор

Сначала получите поражение, затем начните заново и получите победу. В обоих случаях управление блокируется. После повторного запуска все объекты и показатели возвращаются в исходное состояние.

06 / Адаптация без разрушения основы

Как работать со своим вариантом

1

Сохраните общую основу

Оставьте движение, столкновения, состояние игры, панель результата и повторный запуск. Меняйте данные и правила постепенно. После каждого изменения запускайте одну короткую проверку.

2

Измените предметную область

Переименуйте игровые объекты и сообщения, задайте подходящие формы и расположение. Затем реализуйте отличающее правило карточки. Нельзя сдавать «Кристальный маршрут» с заменённым заголовком.

3

Минимум самостоятельного решения

В программе должно появиться хотя бы одно новое состояние или проверка: тип предмета, ожидаемый номер, переносимый объект, заполнение, набор счётчиков либо флаг доступа. Объясните это изменение в отчёте.

4

Если возникла ошибка

Вернитесь к последнему работающему состоянию: отмените только последнее изменение, сравните имя файла и класса, прочитайте первую красную строку Console, исправьте одну причину и снова нажмите Play.

Результат и порядок сдачи

1

Что должно получиться

Один проект своей предметной области: рабочая сцена Main, экранное управление, индивидуальное правило, наблюдаемый прогресс, победа, поражение и полный повтор. Графика по желанию не заменяет программного правила.

2

1. Подготовьте папку для репозитория

Сохраните сцену Main и закройте Unity. В отдельную пустую папку скопируйте Assets целиком вместе с .meta, Packages, ProjectSettings и README.md. Эти четыре элемента должны лежать на верхнем уровне. В README запишите номер варианта, версию Unity, путь Assets/Scenes/Main.unity, управление и условия завершения. Временные папки Library, Temp, Logs, obj, UserSettings и Build в эту копию не входят.

3

2. Загрузите проект и проверьте ссылку

Войдите в GitHub, нажмите «+» → New repository, задайте имя PM12_VariantNN и создайте репозиторий. В пустом репозитории выберите uploading an existing file (или Add file → Upload files), перетащите содержимое подготовленной папки и нажмите Commit changes. Проверьте, что на первой странице видны Assets, Packages, ProjectSettings и README.md. Откройте Assets → Scenes → Main.unity и убедитесь, что файл есть. Если репозиторий закрытый, предоставьте руководителю доступ.

4

3. Проверьте то, что получит проверяющий

В своём репозитории выберите Code → Download ZIP. Распакуйте ZIP в отдельную папку и добавьте через Unity Hub → Projects → Add папку, где рядом лежат Assets, Packages и ProjectSettings. Откройте Main и выполните проверки из раздела «Проверочные сценарии». Запишите ожидания и фактические результаты, включая повтор. Вставьте адрес репозитория в поле «Ссылка на репозиторий проекта» в Word-отчёте.

5

4. Прикрепите комплект в поле «Ответ»

Откройте практику ПМ.12 в личном кабинете. Приложите четыре файла из перечня в пояснительной записке: подписанное на последней странице индивидуальное задание в PDF, презентацию по Приложению 1 в PDF, отчёт о выполнении проекта в DOCX и аттестационный лист в DOCX. В аттестационном листе заполните только первый абзац со своими данными. Сохраните ответ и убедитесь, что отображаются все четыре вложения.

ПМ.12 / Прототип мобильной игры

Файлы для работы

1

Скачать список предметных областей (PDF)

Скачать список предметных областей (PDF)

Найдите карточку по своему порядковому номеру в списке группы и выпишите условия. Отдельных архивов вариантов нет.

2

Скачать тестовый проект «Кристальный маршрут» (ZIP)

Скачать тестовый проект «Кристальный маршрут» (ZIP)

Необязательная контрольная копия для сравнения после проверки своей основы. Открывайте её в отдельной папке, не поверх своей работы. Индивидуальное правило в ней не выполнено.

3

Скачать форму «Отчет о выполнении.docx»

Скачать форму «Отчет о выполнении.docx»

Это форма «Отчёт о выполнении проекта» из комплекта практики. Заполняйте её по мере работы своими действиями, снимками и результатами.

ПМ.12 / Создание, объяснение и проверка

Как устроена работа

1

Один маршрут в своём проекте

В проекте PM12_VariantNN сначала проверим движение, затем контакты и завершение, после этого реализуем правило своей карточки. Код «Кристального маршрута» — разобранный образец общей основы в том же рабочем проекте. Вторую игру для сдачи создавать не нужно.

2

Контрольные точки

Сначала работают три файла и движение. Затем добавляются взаимодействия и полный GameManager. Проверяем общую игру, только после этого переносим механику на вариант. Когда фрагмент временно зависит от следующего, это прямо указано: не пытайтесь исправлять ожидаемые ошибки незаконченного блока.

3

Тестовый проект для сверки

Тестовый проект для сверки

Если собственная основа ведёт себя иначе, откройте тестовый проект в отдельной папке и сравните нужный файл или состояние. Он не является обязательным вторым результатом и не содержит решения индивидуального правила.

Этап 1 / Постановка задачи

Сначала определите правила

1

1. Откройте карточку варианта

Сверьте номер своей строки в списке группы с номером карточки в PDF «Список предметных областей». В README запишите номер, название, сюжет и цель. Если вы стоите в строке 7, откройте вариант 07.

2

2. Выпишите объекты

Составьте три списка: игрок; целевые объекты; опасности или ограничения. Не добавляйте объекты, которые не влияют на правило. Проверка: у каждого объекта записано хотя бы одно действие.

3

3. Запишите условия

Запишите одной фразой условие победы и другой фразой условие поражения. Используйте измеримые значения: количество, флаги, порядок или запас. Не используйте фразу «пройти игру» без условия.

4

Где можно принять своё решение

В карточке заданы обязательные объекты, числа и условия. Если она не задаёт названия, внешний вид, координаты или начальный запас, выберите их и запишите в README. Для жизней или запаса можно начать с 3 и отнимать 1 за ошибку. Разместите достаточно объектов для победы и предусмотрите возможность проверить поражение. Если победа задана последним правильным действием, проверка раннего завершения означает попытку выиграть до этого действия.

5

4. Подготовьте две проверки

Запишите одно правильное и одно неправильное действие для проверки особого правила. Эти действия позднее станут тестом программы.

Перед этапом 2 / Проверить до начала

Версия Unity и рабочие окна

1

Скачать Unity Hub

Скачать Unity Hub

Если Unity Hub ещё нет, скачайте его и установите. Запустите Hub. Если он попросит войти и активировать лицензию, выполните подсказки в Hub или используйте учебный компьютер с подготовленной средой.

2

Открыть страницу Unity 6000.6.2f1

Открыть страницу Unity 6000.6.2f1

Откройте Unity Hub → Installs. Если строки 6000.6.2f1 нет, откройте страницу этой версии по ссылке и выберите Install, затем подтвердите открытие Unity Hub. Дождитесь появления 6000.6.2f1 в Installs. Android Build Support для обязательной проверки в редакторе не нужен.

3

2. Выберите шаблон

В Unity Hub нажмите New project. Слева выберите Unity 6.6 (6000.6.2f1), затем карточку Universal 2D или 2D Core. Не выбирайте 3D: в инструкции используются Rigidbody2D, Collider2D и SpriteRenderer.

4

3. Верните стандартную раскладку

В открытом Unity Editor нажмите Layout в правом верхнем углу → Default. Если Layout не видно, выберите Window → Layouts → Default. После этого слева должна быть Hierarchy, в центре Scene/Game, снизу Project/Console, справа Inspector.

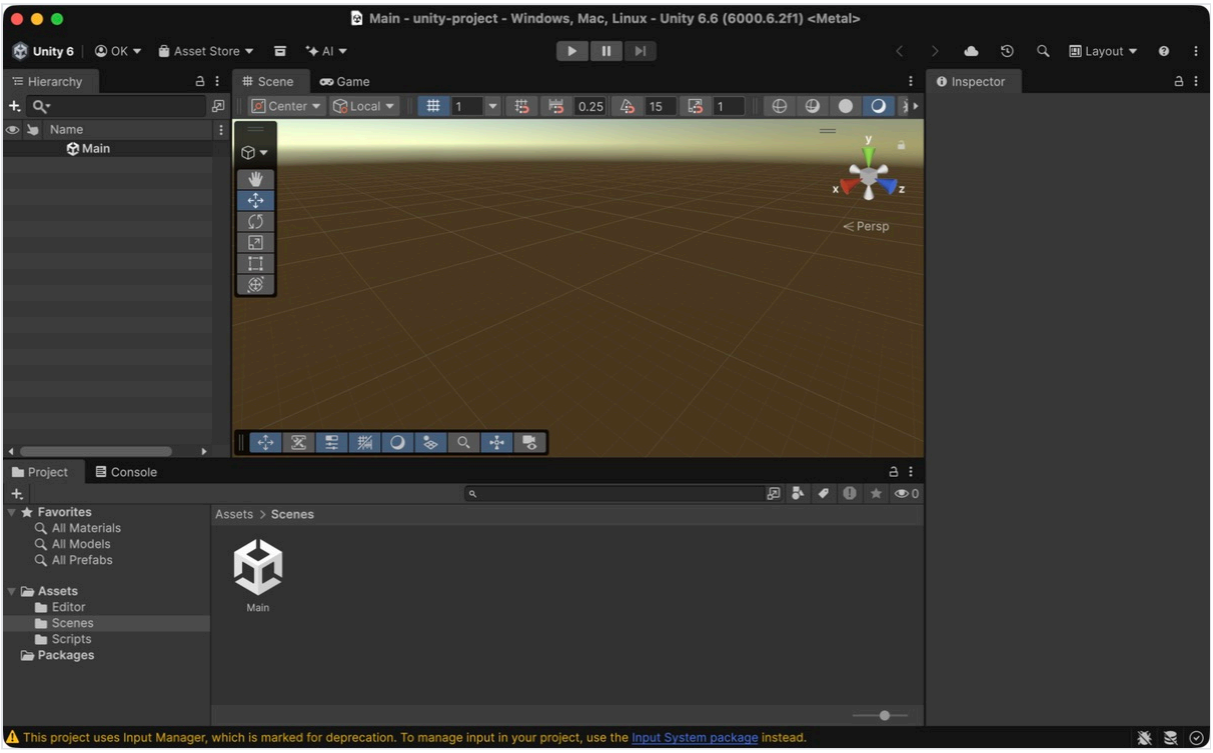
5

4. Назначение окон

Hierarchy показывает объекты сцены. Scene нужна для редактирования. Game показывает результат запуска. Project хранит сцены и скрипты. Inspector показывает компоненты выделенного объекта. Console показывает ошибки. Не переходите дальше, пока не найдёте все шесть областей.

Перед этапом 2 / Сверьте экран

Где находятся панели Unity



Слева — Hierarchy; по центру — вкладки Scene и Game; справа — Inspector; снизу — Project и Console; кнопка Play находится по центру верхней панели.

Этап 2 / Unity · проект и сцена

Подготовьте проект

1

До первого действия

Убедитесь, что в Unity Hub → Installs отображается 6000.6.2f1. Если установить редактор или открыть его не удалось, воспользуйтесь подготовленным учебным компьютером. Откройте Console после создания проекта и переходите к коду, когда в ней нет красных ошибок.

2

1. Создайте проект

Откройте Unity Hub. В левом меню нажмите Projects, затем справа сверху New project. Выберите редактор 6000.6.2f1 и шаблон Universal 2D или 2D Core. В поле Project name введите PM12_VariantNN, заменив NN своим номером. Нажмите кнопку выбора папки, укажите путь без русских букв и нажмите Create project.

3

Проверка 2.1

Открылся Unity Editor. Слева видно окно Hierarchy, в центре Scene, снизу Project, справа Inspector. Если редактор предлагает открыть Safe Mode, сначала откройте Console и исправьте красные ошибки.

4

2. Сохраните сцену

Выберите File → Save As. Создайте папку Assets/Scenes. В поле имени введите Main и нажмите Save. Проверка: в заголовке Unity и в Project видно имя Main.

5

3. Создайте папку кода

В Project выделите Assets. Нажмите правой кнопкой → Create → Folder. Введите Scripts и нажмите Enter. Проверка: путь внизу окна заканчивается на Assets/Scripts после двойного щелчка по папке.

Проверьте редактор кода и систему ввода

1

Редактор скриптов

В Unity откройте Unity → Settings на macOS или Edit → Preferences на Windows, затем External Tools. В External Script Editor выберите уже установленный редактор C#. Если VS Code не установлен, используйте доступный редактор; не требуется устанавливать другой ради примера.

2

Система ввода

Откройте Edit → Project Settings → Player → Other Settings → Configuration → Active Input Handling. Для кода Input.GetKey выберите Input Manager (Old), либо Both, если проекту также нужен Input System. Подтвердите перезапуск редактора, если он запрошен. Это настройка текущего учебного проекта. Если поле свёрнуто, раскройте Other Settings и Configuration.

3

Почему это нужно

Unity поддерживает две системы ввода. В примере используются UnityEngine.Input.GetKey и GetKeyDown старой системы. Только Input System Package (New) не подходит этому коду. При InvalidOperationException прочитайте сообщение и повторно проверьте Active Input Handling.

4

Размер окна проверки

Вкладка Game → список разрешений → 16:9. Если строки нет, нажмите +, выберите Aspect Ratio, Width 16, Height 9 и сохраните. Демонстрация использует горизонтальное поле; портретный экран требует отдельной переработки камеры и интерфейса.

5

Проверка

После перезапуска Main открыта. Дважды щёлкните новый скрипт: открывается выбранный редактор. Пока код не введён, Play не требуется.

ПМ.12 / Создание, объяснение и проверка

Создайте только три файла движения

1

Добавление файлов

В Assets/Scripts щёлкните правой кнопкой по пустому месту → Create → Scripting → MonoBehaviour Script. Создайте по очереди GameBootstrap, PlayerController, GameManager. Если меню версии показывает MonoBehaviour Script сразу в Create, выберите его. Не создавайте Collectible, Hazard и ExitZone до следующей контрольной точки.

2

Замена шаблона

Откройте GameBootstrap двойным щелчком; Ctrl+A на Windows или Cmd+A на macOS удаляет шаблон. Введите приведённый полный код. Сохраните. Аналогично заполните PlayerController. Ни один из этих скриптов не требуется перетаскивать на сцену.

3

Зависимость блока

До заполнения временного GameManager ошибки о GameManager ожидаемы. Завершите все три файла, затем проверяйте Console. Избегайте второго определения одноимённого класса.

GameBootstrap.cs / Запуск и данные

1

Куда вводить

Файл Assets/Scripts/GameBootstrap.cs. Замените весь шаблон этим началом. До последней части и готовности GameManager этот блок ещё не компилируется.

2

Код GameBootstrap.cs

```
using UnityEngine;

public static class GameBootstrap
{
    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
    private static void CreateGame()
    {
        if (Object.FindAnyObjectByType<GameManager>() != null)
            return;

        var root = new GameObject("CrystalQuest");
        root.AddComponent<GameManager>();
    }
}
```

3

Зачем и как работает

Атрибут RuntimeInitializeOnLoadMethod запускает метод после загрузки сцены. Поиск существующего менеджера предотвращает его второе создание. AddComponent вызывает Awake менеджера; вручную подключать Bootstrap не нужно.

4

Сохранение

Сохраните файл. Не запускайте промежуточный незаконченный класс. Финальную проверку движения выполните после следующего файла.

ПМ.12 / Создание, объяснение и проверка

PlayerController.cs / Запуск и данные

1

Куда вводить

Файл Assets/Scripts/PlayerController.cs. Замените весь шаблон этим началом. До последней части и готовности GameManager этот блок ещё не компилируется.

2

Код PlayerController.cs

```
using UnityEngine;

[RequireComponent(typeof(Rigidbody2D), typeof(Collider2D))]
public class PlayerController : MonoBehaviour
{
    [SerializeField, Min(1f)] private float speed = 5f;
    private Rigidbody2D body;
    private Vector2 startPosition;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        body.gravityScale = 0f;
        body.freezeRotation = true;
        startPosition = transform.position;
    }
```

3

Зачем и как работает

RequireComponent гарантирует физические компоненты. Awake получает Rigidbody2D, отключает гравитацию для вида сверху, запрещает вращение и запоминает исходную позицию.

4

Сохранение

Сохраните файл. Не запускайте промежуточный незаконченный класс. Финальную проверку движения выполните после следующего файла.

ПМ.12 / Создание, объяснение и проверка

PlayerController.cs / Движение

1

Куда вводить

Файл Assets/Scripts/PlayerController.cs. Добавьте сразу после предыдущего фрагмента этого файла; не повторяйте уже введённые строки. До последней части и готовности GameManager этот блок ещё не компилируется.

2

Код PlayerController.cs

```
    private void FixedUpdate()
    {
        if (GameManager.Instance == null || !GameManager.Instance.IsPlaying)
        {
            body.linearVelocity = Vector2.zero;
            return;
        }

        Vector2 keyboard = new Vector2(
            ReadAxis(KeyCode.A, KeyCode.LeftArrow, KeyCode.D,
KeyCode.RightArrow),
            ReadAxis(KeyCode.S, KeyCode.DownArrow, KeyCode.W, KeyCode.UpArrow));
        Vector2 direction = Vector2.ClampMagnitude(
            keyboard + GameManager.Instance.MobileDirection, 1f);
        body.linearVelocity = direction * speed;
    }

    private static float ReadAxis(
        KeyCode negativeA, KeyCode negativeB,
```

3

Зачем и как работает

FixedUpdate выполняется с физическим тактом. Когда игра завершена, скорость равна нулю. Два направления суммируются, ClampMagnitude ограничивает длину вектора единицей: диагональное движение не становится быстрее.

4

Сохранение

Сохраните файл. Не запускайте промежуточный незаконченный класс. Финальную проверку движения выполните после следующего файла.

PlayerController.cs / Возврат

1

Куда вводить

Файл Assets/Scripts/PlayerController.cs. Добавьте сразу после предыдущего фрагмента этого файла; не повторяйте уже введённые строки. До последней части и готовности GameManager этот блок ещё не компилируется.

2

Код PlayerController.cs

```
        KeyCode positiveA, KeyCode positiveB)
    {
        float value = 0f;
        if (Input.GetKey(negativeA) || Input.GetKey(negativeB)) value -= 1f;
        if (Input.GetKey(positiveA) || Input.GetKey(positiveB)) value += 1f;
        return value;
    }

    public void ReturnToStart()
    {
        body.linearVelocity = Vector2.zero;
        body.position = startPosition;
    }
}
```

3

Зачем и как работает

ReadAxis складывает отрицательное и положительное направления: противоположные клавиши компенсируются. ReturnToStart обнуляет скорость и возвращает тело в исходную позицию.

4

Сохранение

Сохраните файл. Не запускайте промежуточный незаконченный класс. Финальную проверку движения выполните после следующего файла.

Завершите основу движения

1

Источник и место

Откройте Assets/Scripts/GameManager.cs. Полностью замените шаблон следующим кодом. Это первая учебная версия менеджера: она создаёт камеру и игрока, но пока не содержит предметов и победы. Не добавляйте рядом второй GameManager.

2

Напишите временный GameManager.cs для проверки движения

```
using UnityEngine;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }
    public bool IsPlaying => true;
    public Vector2 MobileDirection { get; private set; }
    private float mobileInputUntil;

    private void Awake()
    {
        Instance = this;
        Camera camera = Camera.main;
        if (camera == null)
        {
            var cameraObject = new GameObject("Main Camera");
            camera = cameraObject.AddComponent<Camera>();
            cameraObject.tag = "MainCamera";
        }
        camera.orthographic = true;
        camera.orthographicSize = 5.3f;
        camera.transform.position = new Vector3(0, 0, -10);
        var player = new GameObject("Player");
        var texture = new Texture2D(32, 32);
        for (int y = 0; y < 32; y++)
```

3

Напишите временный GameManager.cs для проверки движения · продолжение

```
            for (int x = 0; x < 32; x++) texture.SetPixel(x, y, Color.cyan);
        texture.Apply();
        var renderer = player.AddComponent<SpriteRenderer>();
        renderer.sprite = Sprite.Create(texture, new Rect(0, 0, 32, 32),
            new Vector2(0.5f, 0.5f), 32);
        player.AddComponent<Rigidbody2D>();
        player.AddComponent<BoxCollider2D>();
        player.AddComponent<PlayerController>();
    }

    private void OnGUI()
    {
        Vector2 direction = Vector2.zero;
        if (GUI.RepeatButton(new Rect(90, 20, 70, 70), "UP")) direction.y++;
        if (GUI.RepeatButton(new Rect(90, 100, 70, 70), "DOWN")) direction.y--;
        if (GUI.RepeatButton(new Rect(10, 100, 70, 70), "LEFT")) direction.x--;
        if (GUI.RepeatButton(new Rect(170, 100, 70, 70), "RIGHT"))
direction.x++;
        if (direction != Vector2.zero)
        {
            MobileDirection = direction;
```

4

Напишите временный GameManager.cs для проверки движения · продолжение

```
            mobileInputUntil = Time.time + 0.08f;
        }
        else if (Time.time >= mobileInputUntil) MobileDirection = Vector2.zero;
    }
}
```

5

Связь компонентов

Awake присваивает Instance, чтобы PlayerController мог проверить IsPlaying. AddComponent подключает физику и управление созданному игроку. OnGUI собирает экранное направление; клавиатурный ввод читает PlayerController. При вызове AddComponent Awake игрока выполняется сразу. Направление экранной кнопки сохраняется на 0.08 секунды, чтобы короткое GUI-нажатие успело попасть в FixedUpdate. Удержание продлевает ввод; после отпускания он прекращается.

Проверьте первый работающий результат

1

Запуск

File → Save All в редакторе кода. В Unity дождитесь компиляции, Window → General → Console → Clear. Красных сообщений быть не должно. Откройте Main и нажмите Play, затем Game.

2

Действия и ожидание

Виден голубой квадрат. WASD и стрелки двигают его по четырём направлениям. Удержание каждой экранной кнопки двигает в указанную сторону. При отпускании движение прекращается. Стен ещё нет: уход за экран в этой версии допустим. Остановите Play.

3

Диагностика

Ошибка ввода: проверьте Active Input Handling. Нет игрока: проверьте имена классов и наличие GameBootstrap; Console покажет первую причину. Падение вниз: убедитесь, что Awake PlayerController задаёт gravityScale = 0. Не переходите к сбору предметов, пока движение не работает.

ПМ.12 / Создание, объяснение и проверка

Подготовьте контакты и полный менеджер

1

Добавьте оставшиеся файлы

В Assets/Scripts создайте Collectible, Hazard и ExitZone через то же меню MonoBehaviour Script. Заполните их полностью по следующему разбору. Они вызывают методы полной версии GameManager, поэтому до её завершения ожидаются ошибки отсутствующих методов.

2

Что такое триггер

Collider2D задаёт область контакта. У стены Is Trigger выключен: она останавливает игрока. У предмета, опасности и выхода Is Trigger включён: они сообщают о входе игрока, но не перекрывают путь. Rigidbody2D у игрока обеспечивает физическую обработку этих контактов.

ПМ.12 / Создание, объяснение и проверка

Collectible.cs / Контакт с игроком

1

Куда вводить

В Assets/Scripts/Collectible.cs замените шаблон целиком. После сохранения переходите к следующему файлу; методы полного менеджера ещё не введены.

2

Полный компонент контакта

```
using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class Collectible : MonoBehaviour
{
    private Vector3 startScale;

    private void Awake()
    {
        GetComponent<Collider2D>().isTrigger = true;
        startScale = transform.localScale;
    }

    private void Update()
    {
        transform.Rotate(0f, 0f, 70f * Time.deltaTime);
        float pulse = 1f + Mathf.Sin(Time.time * 4f) * 0.08f;
        transform.localScale = startScale * pulse;
    }
```

3

Полный компонент контакта · продолжение

```
    private void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.TryGetComponent<PlayerController>(out _)) return;
        GameManager.Instance.Collect(this);
    }
}
```

4

Почему так

Awake делает коллайдер триггером и сохраняет масштаб. Update вращает и пульсирует изображение; Time.deltaTime делает скорость независимой от числа кадров. OnTriggerEnter2D отбирает именно PlayerController, затем передаёт предмет менеджеру. Сам предмет не решает, наступила ли победа.

ПМ.12 / Создание, объяснение и проверка

Hazard.cs / Контакт с игроком

1

Куда вводить

В Assets/Scripts/Hazard.cs замените шаблон целиком. После сохранения переходите к следующему файлу; методы полного менеджера ещё не введены.

2

Полный компонент контакта

```
using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class Hazard : MonoBehaviour
{
    private void Awake()
    {
        GetComponent<Collider2D>().isTrigger = true;
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.TryGetComponent<PlayerController>(out _)) return;
        GameManager.Instance.HitHazard();
    }
}
```

3

Почему так

Триггер опасности проверяет вход игрока и вызывает HitHazard. Жизни и паузу между повреждениями хранит один менеджер: две опасности не должны независимо создавать несовместимые счётчики.

ПМ.12 / Создание, объяснение и проверка

ExitZone.cs / Контакт с игроком

1

Куда вводить

В Assets/Scripts/ExitZone.cs замените шаблон целиком. После сохранения переходите к следующему файлу; методы полного менеджера ещё не введены.

2

Полный компонент контакта

```
using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class ExitZone : MonoBehaviour
{
    private void Awake()
    {
        GetComponent<Collider2D>().isTrigger = true;
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.TryGetComponent<PlayerController>(out _)) return;
        GameManager.Instance.TryFinishLevel();
    }
}
```

3

Почему так

Выход вызывает TryFinishLevel. Решение принимается по состоянию всей игры, поэтому ранний вход в портал не должен автоматически завершать уровень.

ПМ.12 / Создание, объяснение и проверка

Начните полную версию GameManager

1

Как заменить временную версию

В Assets/Scripts/GameManager.cs выделите весь временный код и удалите его. Далее вводите логические блоки в одном файле в указанном порядке. Первое объявление открывает класс; последняя часть закрывает его. Не оставляйте временный Awake или OnGUI рядом с новой версией.

2

Зависимости

До последнего блока методы ссылаются на ещё не добавленные методы и Shape. Эти сообщения ожидаемы. В этот период сохраняйте код, но не выполняйте Play. Мы расширяем уже проверенную основу движения; следующая рабочая контрольная точка — весь полный менеджер.

ПМ.12 / Создание, объяснение и проверка

GameManager / Данные

1

Место и действие

Assets/Scripts/GameManager.cs: начало файла, замените временный код. Пока полный класс не завершён, не запускайте игру.

2

Логический блок Данные

```
using System.Collections.Generic;
using System.IO;
using UnityEngine;

public class GameManager : MonoBehaviour
{
    private enum GameState { Playing, Won, Lost }

    public static GameManager Instance { get; private set; }
    public bool IsPlaying => state == GameState.Playing;
    public Vector2 MobileDirection { get; private set; }

    private const int Target = 5;
    private readonly List<GameObject> createdObjects = new();
    private readonly Vector2[] crystalPositions =
    {
        new(-5.7f, 2.8f), new(-2.5f, 0.8f), new(0.8f, 3.0f),
        new(3.2f, -1.8f), new(5.7f, 2.2f)
    };
    private readonly Vector2[] hazardPositions =
    {
        new(-4.0f, -1.8f), new(-0.6f, -0.2f), new(2.2f, 1.5f),
        new(5.0f, -0.5f)
    };
```

3

Логический блок Данные · продолжение

```
    private PlayerController player;
    private GameState state;
    private int collected;
    private int hearts;
    private float invulnerableUntil;
    private float mobileInputUntil;
    private float hintUntil;
    private string hint = "";
    private GUIStyle titleStyle;
    private GUIStyle textStyle;
    private GUIStyle centerStyle;
    private GUIStyle buttonStyle;
    private Texture2D panelTexture;
    private Texture2D buttonTexture;
    private Texture2D buttonPressedTexture;
```

4

Назначение и связь

GameState перечисляет Playing, Won и Lost. Target задаёт число предметов, массивы позиций задают данные уровня, Instance связывает компоненты с менеджером. Поля хранят игрока, счётчики, сроки подсказки и оформление. Один источник состояния предотвращает расхождение интерфейса и правил.

ПМ.12 / Создание, объяснение и проверка

GameManager / Awake

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 42–49 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок Awake

```
    private void Awake()
    {
        Instance = this;
        Application.targetFrameRate = 60;
        BuildLevel();
        RestartGame();
    }
```

3

Назначение и связь

Awake связывает Instance, задаёт частоту кадров и создаёт уровень. Затем RestartGame задаёт начальное состояние. Создание уровня и сброс разделены: при повторе стены и камера не дублируются.

ПМ.12 / Создание, объяснение и проверка

GameManager / Update

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 50–60 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок Update

```
    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.R)) RestartGame();

        // Клавиши F6-F9 нужны преподавателю для быстрой демонстрации и снимков.
        if (Input.GetKeyDown(KeyCode.F6) && IsPlaying) DebugCollectOne();
        if (Input.GetKeyDown(KeyCode.F7)) DebugSetState(true);
        if (Input.GetKeyDown(KeyCode.F8)) DebugSetState(false);
        if (Input.GetKeyDown(KeyCode.F9)) CaptureScreenshot();
    }
```

3

Назначение и связь

Update читает команду R. F6–F9 предназначены для демонстрации и снимков: они не заменяют игровую проверку контактов. При самостоятельной проверке получайте победу сбором и входом в выход.

GameManager / BuildLevel

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 61–91 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок BuildLevel

```
    private void BuildLevel()
    {
        CreateCamera();
        CreateDecor();
        CreateWalls();

        var playerObject = CreateSpriteObject(
            "Player", new Vector2(-6.5f, -3.1f), new Vector2(0.75f, 0.75f),
            new Color(0.19f, 0.88f, 1f), Shape.Circle, 10);
        playerObject.AddComponent<Rigidbody2D>();
        playerObject.AddComponent<CircleCollider2D>();
        player = playerObject.AddComponent<PlayerController>();

        foreach (Vector2 position in hazardPositions)
        {
            var hazard = CreateSpriteObject(
                "Hazard", position, new Vector2(0.85f, 0.85f),
                new Color(1f, 0.25f, 0.28f), Shape.Cross, 4);
            hazard.AddComponent<CircleCollider2D>();
            hazard.AddComponent<Hazard>();
        }
```

3

Логический блок BuildLevel · продолжение

```
        var exit = CreateSpriteObject(
            "Exit", new Vector2(6.5f, 3.15f), new Vector2(1.2f, 1.2f),
            new Color(0.25f, 0.82f, 0.45f), Shape.Exit, 2);
        exit.AddComponent<BoxCollider2D>();
        exit.AddComponent<ExitZone>();

        CreateCrystals();
    }
```

4

Назначение и связь

BuildLevel создаёт игрока и физику, затем опасности и выход. Каждый объект получает SpriteRenderer, Collider2D и свой обработчик. AddComponent<PlayerController> выполняется после добавления физического тела.

ПМ.12 / Создание, объяснение и проверка

GameManager / CreateCamera

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 92–106 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок CreateCamera

```
private void CreateCamera()
{
    Camera camera = Camera.main;
    if (camera == null)
    {
        var cameraObject = new GameObject("Main Camera");
        cameraObject.tag = "MainCamera";
        camera = cameraObject.AddComponent<Camera>();
    }
    camera.orthographic = true;
    camera.orthographicSize = 5.3f;
    camera.transform.position = new Vector3(0f, 0f, -10f);
    camera.backgroundColor = new Color(0.035f, 0.055f, 0.10f);
}
```

3

Назначение и связь

Камера ортографическая: расстояние до камеры не меняет размер объекта. Size 5.3 соответствует половине видимой высоты. В окне 16:9 поле помещается целиком.

ПМ.12 / Создание, объяснение и проверка

GameManager / CreateDecor

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 107–116 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок CreateDecor

```
    private void CreateDecor()
    {
        CreateSpriteObject("Field", Vector2.zero, new Vector2(16.4f, 9.2f),
            new Color(0.07f, 0.10f, 0.18f), Shape.Square, -10);
        for (int x = -7; x <= 7; x += 2)
            for (int y = -4; y <= 4; y += 2)
                CreateSpriteObject("FloorDot", new Vector2(x, y), Vector2.one *
0.06f,
                    new Color(0.18f, 0.23f, 0.34f), Shape.Circle, -9);
    }
```

3

Назначение и связь

Фон и точки не имеют Collider2D: это декорации. Слой -10 помещает поле под игровые объекты. Вложенные циклы рисуют сетку равномерно.

ПМ.12 / Создание, объяснение и проверка

GameManager / CreateWalls

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 117–127 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок CreateWalls

```
    private void CreateWalls()
    {
        CreateWall("WallTop", new Vector2(0f, 4.65f), new Vector2(17f, 0.3f));
        CreateWall("WallBottom", new Vector2(0f, -4.65f), new Vector2(17f,
0.3f));
        CreateWall("WallLeft", new Vector2(-8.35f, 0f), new Vector2(0.3f,
9.6f));
        CreateWall("WallRight", new Vector2(8.35f, 0f), new Vector2(0.3f,
9.6f));
        CreateWall("WallA", new Vector2(-3.8f, 1.2f), new Vector2(0.35f, 3.0f));
        CreateWall("WallB", new Vector2(1.4f, -2.2f), new Vector2(3.2f, 0.35f));
        CreateWall("WallC", new Vector2(4.4f, 2.0f), new Vector2(0.35f, 2.5f));
    }
```

3

Назначение и связь

Четыре внешние стены удерживают игрока в поле. Три внутренние создают маршрут. Position задаёт центр, scale — ширину и высоту. Не ставьте предмет внутрь стены.

ПМ.12 / Создание, объяснение и проверка

GameManager / CreateWall

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 128–134 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок CreateWall

```
private void CreateWall(string name, Vector2 position, Vector2 scale)
{
    var wall = CreateSpriteObject(name, position, scale,
        new Color(0.20f, 0.27f, 0.40f), Shape.Square, 1);
    wall.AddComponent<BoxCollider2D>();
}
```

3

Назначение и связь

Каждой стене нужен BoxCollider2D без Is Trigger. Rigidbody2D ей не требуется: она неподвижная. CreateSpriteObject возвращает объект, чтобы затем можно было добавить компонент.

ПМ.12 / Создание, объяснение и проверка

GameManager / CreateCrystals

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 135–147 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок CreateCrystals

```
private void CreateCrystals()
{
    foreach (Vector2 position in crystalPositions)
    {
        var crystal = CreateSpriteObject(
            "Crystal", position, new Vector2(0.62f, 0.82f),
            new Color(1f, 0.78f, 0.16f), Shape.Diamond, 5);
        crystal.AddComponent<PolygonCollider2D>();
        crystal.AddComponent<Collectible>();
        createdObjects.Add(crystal);
    }
}
```

3

Назначение и связь

foreach создаёт предмет для каждой позиции. PolygonCollider2D подстраивается под спрайт, Collectible включает триггер. createdObjects хранит ещё существующие предметы для корректного повторного запуска.

ПМ.12 / Создание, объяснение и проверка

GameManager / CreateSpriteObject

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 148–160 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок CreateSpriteObject

```
private GameObject CreateSpriteObject(
    string name, Vector2 position, Vector2 scale,
    Color color, Shape shape, int order)
{
    var item = new GameObject(name);
    item.transform.position = position;
    item.transform.localScale = scale;
    var renderer = item.AddComponent<SpriteRenderer>();
    renderer.sprite = CreateSprite(shape, color);
    renderer.sortingOrder = order;
    return item;
}
```

3

Назначение и связь

Общий метод создаёт объект, выставляет Transform, добавляет SpriteRenderer и порядок отображения. Он не подключает логику контакта: вызывающий метод делает это отдельно.

ПМ.12 / Создание, объяснение и проверка

GameManager / CreateSprite

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 161–188 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок CreateSprite

```
    private static Sprite CreateSprite(Shape shape, Color color)
    {
        const int size = 64;
        var texture = new Texture2D(size, size, TextureFormat.RGBA32, false);
        texture.filterMode = FilterMode.Bilinear;
        for (int y = 0; y < size; y++)
        {
            for (int x = 0; x < size; x++)
            {
                float nx = (x + 0.5f) / size * 2f - 1f;
                float ny = (y + 0.5f) / size * 2f - 1f;
                bool inside = shape switch
                {
                    Shape.Circle => nx * nx + ny * ny <= 0.90f,
                    Shape.Diamond => Mathf.Abs(nx) + Mathf.Abs(ny) <= 0.92f,
                    Shape.Cross => Mathf.Abs(nx - ny) < 0.22f || Mathf.Abs(nx +
ny) < 0.22f,
                    Shape.Exit => Mathf.Max(Mathf.Abs(nx), Mathf.Abs(ny)) <=
0.92f &&
                                  Mathf.Min(Mathf.Abs(nx), Mathf.Abs(ny)) >=
0.22f,
                    _ => true
                };
```

3

Логический блок CreateSprite · продолжение

```
                texture.SetPixel(x, y, inside ? color : Color.clear);
            }
        }
        texture.Apply();
        return Sprite.Create(texture, new Rect(0, 0, size, size),
            new Vector2(0.5f, 0.5f), size);
    }
```

4

Назначение и связь

Texture2D хранит пиксели. Два цикла вычисляют координаты относительно центра; switch определяет форму. Прозрачные пиксели позволяют различать круг, ромб и крест. Sprite.Create связывает текстуру с Unity-спрайтом; дополнительных PNG не нужно.

GameManager / Collect

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 189–197 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок Collect

```
    public void Collect(Collectible item)
    {
        if (!IsPlaying || item == null ||
!createdObjects.Contains(item.gameObject)) return;
        collected++;
        createdObjects.Remove(item.gameObject);
        Destroy(item.gameObject);
        ShowHint($"Кристалл собран: {collected} из {Target}");
    }
```

3

Назначение и связь

Контакт засчитывается только во время Playing. Счётчик растёт, предмет удаляется из списка и сцены, затем выводится подсказка. Это место позднее изменяется под правило выбора, порядка или груза.

4

Почему один предмет считается один раз

Destroy откладывает уничтожение Unity-объекта до конца кадра. Сначала Collect проверяет, что объект ещё находится в createdObjects; принятое действие удаляет его из списка сразу. Поэтому повторный вызов с тем же предметом уже не увеличивает счётчик. Отказ в правиле варианта оставляет предмет в списке: его можно попробовать подобрать позже.

ПМ.12 / Создание, объяснение и проверка

GameManager / HitHazard

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 198–211 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок HitHazard

```
public void HitHazard()
{
    if (!IsPlaying || Time.time < invulnerableUntil) return;
    hearts--;
    invulnerableUntil = Time.time + 1f;
    player.ReturnToStart();
    if (hearts <= 0)
    {
        state = GameState.Lost;
        MobileDirection = Vector2.zero;
    }
    else ShowHint("Опасность! Потеряна одна жизнь");
}
```

3

Назначение и связь

Одна жизнь снимается только вне периода защиты. После повреждения игрок возвращается в начало. Нулевое число жизней переводит игру в Lost и обнуляет экранное направление. Паузу проверяйте при быстрых повторных контактах.

GameManager / TryFinishLevel

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 212–222 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок TryFinishLevel

```
    public void TryFinishLevel()
    {
        if (!IsPlaying) return;
        if (collected == Target)
        {
            state = GameState.Won;
            MobileDirection = Vector2.zero;
        }
        else ShowHint($"Сначала соберите все кристаллы: {collected} из
{Target}");
    }
```

3

Назначение и связь

Ранний вход показывает подсказку и сохраняет Playing. Равенство collected и Target переводит в Won. Физическое движение прекращает PlayerController через IsPlaying.

GameManager / ShowHint

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 223–228 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок ShowHint

```
    private void ShowHint(string message)
    {
        hint = message;
        hintUntil = Time.time + 2.2f;
    }
```

3

Назначение и связь

Текст и срок показа хранятся отдельно. Time.time сравнивается с hintUntil в OnGUI: сообщение исчезает без отдельного таймера или потока.

ПМ.12 / Создание, объяснение и проверка

GameManager / RestartGame

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 229–245 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок RestartGame

```
private void RestartGame()
{
    foreach (GameObject item in createdObjects)
        if (item != null) Destroy(item);
    createdObjects.Clear();
    collected = 0;
    invulnerableUntil = 0f;
    mobileInputUntil = 0f;
    hearts = 3;
    state = GameState.Playing;
    MobileDirection = Vector2.zero;
    hint = "Соберите 5 кристаллов и войдите в зелёный портал";
    hintUntil = Time.time + 4f;
    if (player != null) player.ReturnToStart();
    CreateCrystals();
}
```

3

Назначение и связь

Удаляются остатки прежних кристаллов, очищается список, сбрасываются счётчики, защита от урона, направление и состояние. Игрок возвращается, пять предметов создаются заново. Любое добавленное состояние варианта тоже должно обнуляться здесь.

GameManager / DebugCollectOne

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 246–251 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок DebugCollectOne

```
private void DebugCollectOne()
{
    if (createdObjects.Count == 0) return;
    Collect(createdObjects[0].GetComponent<Collectible>());
}
```

3

Назначение и связь

Это служебное действие демонстрации: оно использует тот же Collect. В проверке студента оно не доказывает фактическое срабатывание триггера.

GameManager / DebugSetState

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 252–269 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок DebugSetState

```
private void DebugSetState(bool won)
{
    if (won)
    {
        foreach (GameObject item in new List<GameObject>(createdObjects))
            if (item != null) Destroy(item);
        createdObjects.Clear();
        collected = Target;
        state = GameState.Won;
    }
    else
    {
        hearts = 0;
        state = GameState.Lost;
    }
    MobileDirection = Vector2.zero;
}
```

3

Назначение и связь

Служебная команда устанавливает экран результата без прохождения маршрута. Скриншот такого экрана нельзя считать доказательством правильного условия победы.

ПМ.12 / Создание, объяснение и проверка

GameManager / CaptureScreenshot

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 270–281 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок CaptureScreenshot

```
private void CaptureScreenshot()
{
    string directory = Path.GetFullPath(Path.Combine(Application.dataPath,
        "../Documentation/Screenshots"));
    Directory.CreateDirectory(directory);
    string stateName = state.ToString().ToLowerInvariant();
    string path = Path.Combine(directory,
        $"game-{stateName}-{collected}-of-{Target}.png");
    ScreenCapture.CaptureScreenshot(path, 1);
    ShowHint("Снимок сохранён: " + Path.GetFileName(path));
}
```

3

Назначение и связь

ScreenCapture сохраняет фактический Game View. Имя содержит состояние и прогресс. Не используйте этот метод в самостоятельной мобильной сборке без выбора доступного каталога; для проверки достаточно обычного снимка экрана.

ПМ.12 / Создание, объяснение и проверка

GameManager / CreateGuiStyles

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 282–311 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок CreateGuiStyles

```
    private void CreateGuiStyles()
    {
        if (titleStyle != null) return;
        panelTexture = SolidTexture(new Color(0.055f, 0.075f, 0.13f, 0.96f));
        buttonTexture = SolidTexture(new Color(0.12f, 0.18f, 0.30f, 0.94f));
        buttonPressedTexture = SolidTexture(new Color(0.16f, 0.55f, 0.75f,
0.98f));

        titleStyle = new GUIStyle(GUI.skin.label)
        {
            fontSize = 28, fontStyle = FontStyle.Bold,
            normal = { textColor = Color.white }
        };
        textStyle = new GUIStyle(GUI.skin.label)
        {
            fontSize = 19, normal = { textColor = new Color(0.82f, 0.88f, 0.98f)
}
        };
        centerStyle = new GUIStyle(titleStyle)
        {
            alignment = TextAnchor.MiddleCenter, fontSize = 34
        };
        buttonStyle = new GUIStyle(GUI.skin.button)
        {
```

3

Логический блок CreateGuiStyles · продолжение

```
            fontSize = 25, fontStyle = FontStyle.Bold,
            alignment = TextAnchor.MiddleCenter,
            normal = { background = buttonTexture, textColor = Color.white },
            hover = { background = buttonPressedTexture, textColor = Color.white
},
            active = { background = buttonPressedTexture, textColor =
Color.white }
        };
    }
```

4

Назначение и связь

GUIStyle задаёт размер, цвет и выравнивание надписей. Текстуры кнопок создаются один раз: проверка titleStyle предотвращает лишнее создание при каждом кадре.

ПМ.12 / Создание, объяснение и проверка

GameManager / SolidTexture

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 312–319 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок SolidTexture

```
private static Texture2D SolidTexture(Color color)
{
    var texture = new Texture2D(1, 1);
    texture.SetPixel(0, 0, color);
    texture.Apply();
    return texture;
}
```

3

Назначение и связь

Одна заливка создаёт фон элемента интерфейса. Apply переносит изменения пикселей в текстуру; без него рисунок не обновится.

ПМ.12 / Создание, объяснение и проверка

GameManager / OnGUI

1

Место и действие

Assets/Scripts/GameManager.cs: добавьте после предыдущего блока, перед будущим следующим методом; это строки 320–364 итогового исходника. Пока полный класс не завершён, не запускайте игру.

2

Логический блок OnGUI

```
    private void OnGUI()
    {
        CreateGuiStyles();
        float scale = Mathf.Min(Screen.width / 1280f, Screen.height / 720f);
        GUI.matrix = Matrix4x4.Scale(Vector3.one * scale);

        GUI.DrawTexture(new Rect(24, 20, 1232, 78), panelTexture);
        GUI.Label(new Rect(48, 35, 430, 45), "КРИСТАЛЬНЫЙ МАРШРУТ", titleStyle);
        GUI.Label(new Rect(500, 35, 280, 45), $"Кристаллы:
{collected}/{Target}", textStyle);
        GUI.Label(new Rect(820, 35, 250, 45), $"Жизни: {new string('♥',
hearts)}", textStyle);
        GUI.Label(new Rect(1080, 35, 150, 45), "R — заново", textStyle);
```

3

Логический блок OnGUI · продолжение

```
        Vector2 direction = Vector2.zero;
        if (GUI.RepeatButton(new Rect(110, 540, 72, 72), "↑", buttonStyle))
direction.y += 1f;
        if (GUI.RepeatButton(new Rect(110, 620, 72, 72), "↓", buttonStyle))
direction.y -= 1f;
        if (GUI.RepeatButton(new Rect(30, 620, 72, 72), "←", buttonStyle))
direction.x -= 1f;
        if (GUI.RepeatButton(new Rect(190, 620, 72, 72), "→", buttonStyle))
direction.x += 1f;
        if (!IsPlaying) MobileDirection = Vector2.zero;
        else if (direction != Vector2.zero)
        {
            MobileDirection = Vector2.ClampMagnitude(direction, 1f);
            mobileInputUntil = Time.time + 0.08f;
        }
        else if (Time.time >= mobileInputUntil) MobileDirection = Vector2.zero;
```

4

Логический блок OnGUI · продолжение

```
        if (Time.time < hintUntil)
        {
            GUI.DrawTexture(new Rect(340, 620, 600, 58), panelTexture);
            GUI.Label(new Rect(360, 629, 560, 40), hint, textStyle);
        }

        if (state == GameState.Playing) return;
        GUI.DrawTexture(new Rect(340, 210, 600, 280), panelTexture);
        string heading = state == GameState.Won ? "УРОВЕНЬ ПРОЙДЕН" :
"ПОПРОБУЙТЕ ЕЩЁ";
        string message = state == GameState.Won
            ? "Все кристаллы собраны, портал открыт."
            : "Жизни закончились. Измените маршрут.";
        GUI.Label(new Rect(370, 250, 540, 60), heading, centerStyle);
        GUI.Label(new Rect(405, 330, 470, 45), message, textStyle);
        if (GUI.Button(new Rect(475, 400, 330, 62), "НАЧАТЬ ЗАНОВО",
buttonStyle))
            RestartGame();
    }
```

5

Логический блок OnGUI · продолжение

```
    private enum Shape { Square, Circle, Diamond, Cross, Exit }
}
```

6

Назначение и связь

Интерфейс масштабируется от 1280×720. Rect задаёт координаты и размеры кнопок; RepeatButton работает при удержании. Направление обнуляется перед чтением кнопок, чтобы после отпускания игрок остановился. Playing не рисует панель конца; Won и Lost показывают разный текст и одну команду повтора. Короткое нажатие сохраняет направление на 0.08 секунды: событие GUI и FixedUpdate выполняются в разных циклах, поэтому немедленный сброс мог пропустить ввод. Удержание обновляет срок; отпускание останавливает движение после этого короткого остатка. При завершении и новом запуске направление и срок сбрасываются.

Проверьте общую основу своего проекта

1

Компиляция и запуск

В редакторе File → Save All; Unity → Console → Clear. Дождитесь компиляции. Исправьте первую красную ошибку, если она есть. Откройте Main, нажмите Play, выберите Game 16:9.

2

Начало и движение

Ожидаются пять ромбов, четыре красных креста, три жизни и прогресс 0/5. Проверьте четыре клавиши и экранные кнопки. Диагональ не быстрее прямого пути; через стены пройти нельзя.

3

Контакты и отказ

Соберите один ромб: он исчезает, прогресс 1/5. Коснитесь опасности: потеря одной жизни и возврат в начало. Войдите в портал до пяти предметов: игра продолжает работать и объясняет недостающую цель.

4

Конец и повтор

Получите три отдельных повреждения: Lost, управление остановлено. Нажмите «НАЧАТЬ ЗАНОВО»: 0/5, три жизни, пять предметов. Соберите все предметы игровыми действиями и войдите в выход: Won. Повторите запуск после победы и проверьте исходное состояние.

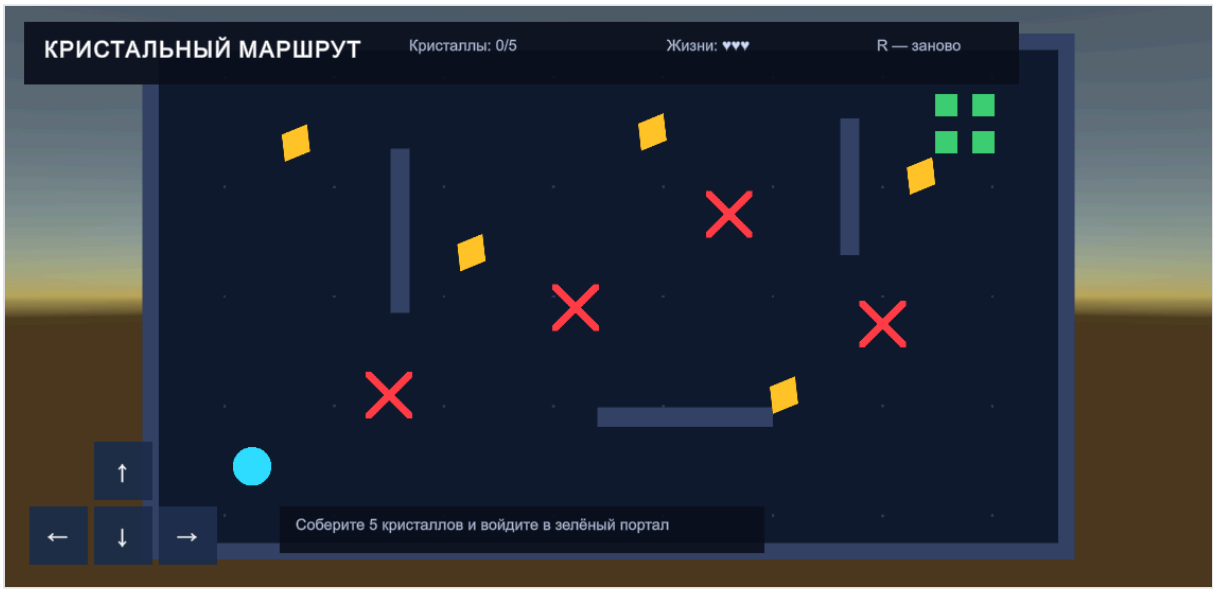
5

Регрессия

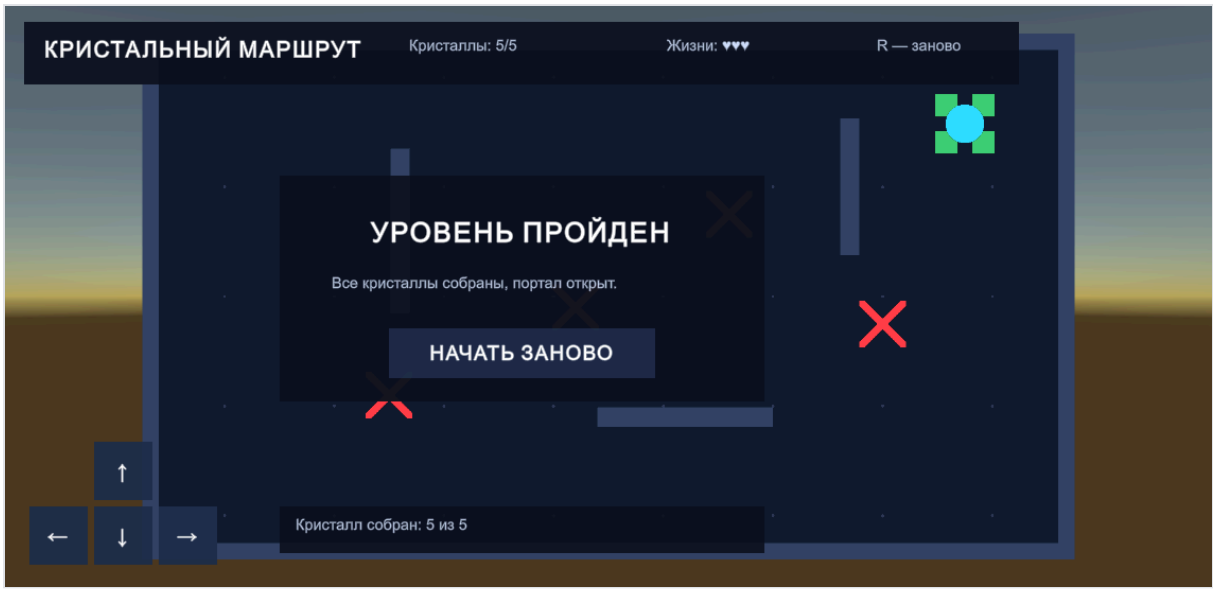
После любого изменения повторяйте движение, стену, правильный контакт, неправильный контакт, ранний финиш, победу, поражение и повтор. Результат проверки описывайте конкретно: действие, ожидание, наблюдение, вывод; одно слово «работает» не объясняет проверку.

ПМ.12 / Проверка реализации

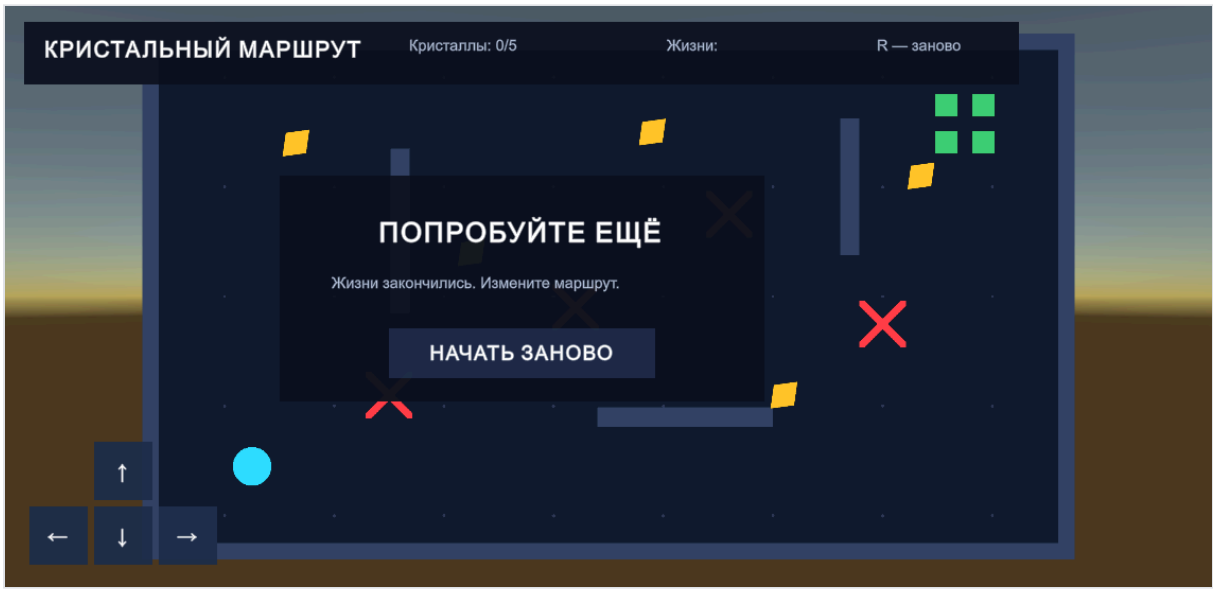
Состояния разобранного образца



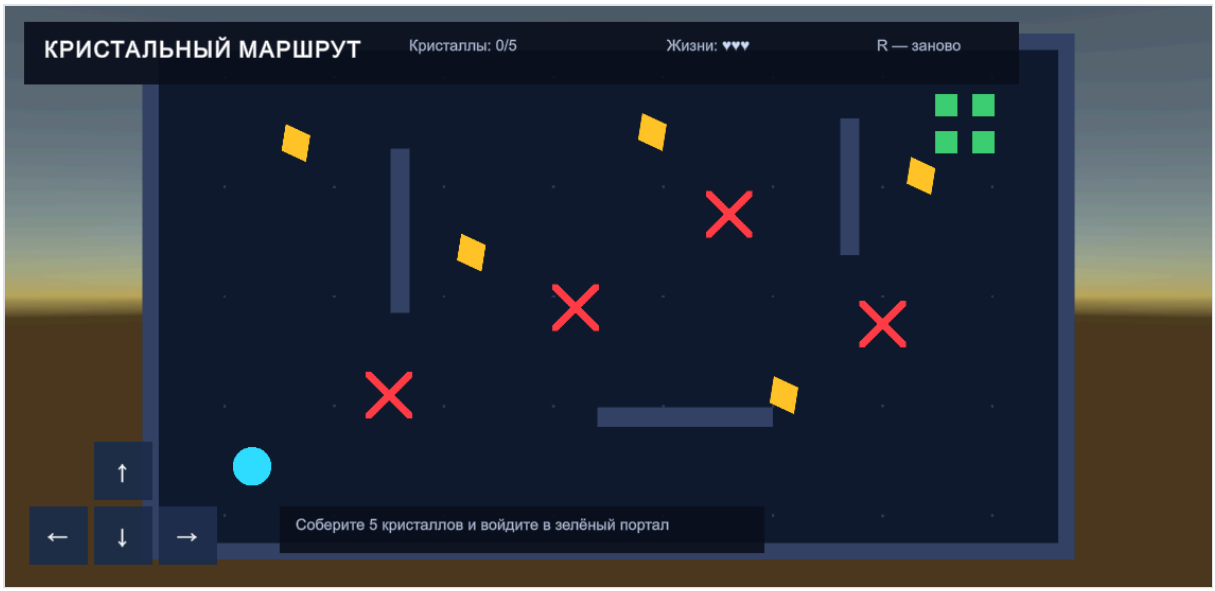
Unity 6000.6.2f1: прогресс 0/5 и три жизни после создания уровня.



Экран получен после пяти контактов игрока с предметами и входа в выход; служебная команда F7 не использовалась.



Экран получен после трёх повреждений с паузой между контактами; F8 не использовалась.



Повтор восстанавливает игрока, пять предметов, прогресс 0/5 и три жизни.

Этап 4 / Две короткие проверки

Проверьте сбор и опасность

1

1. Проверьте кристалл

Нажмите Play. Подведите голубого игрока к одному жёлтому кристаллу. Ожидается: кристалл исчез, сверху написано «Кристаллы: 1/5», снизу появляется короткая подсказка.

2

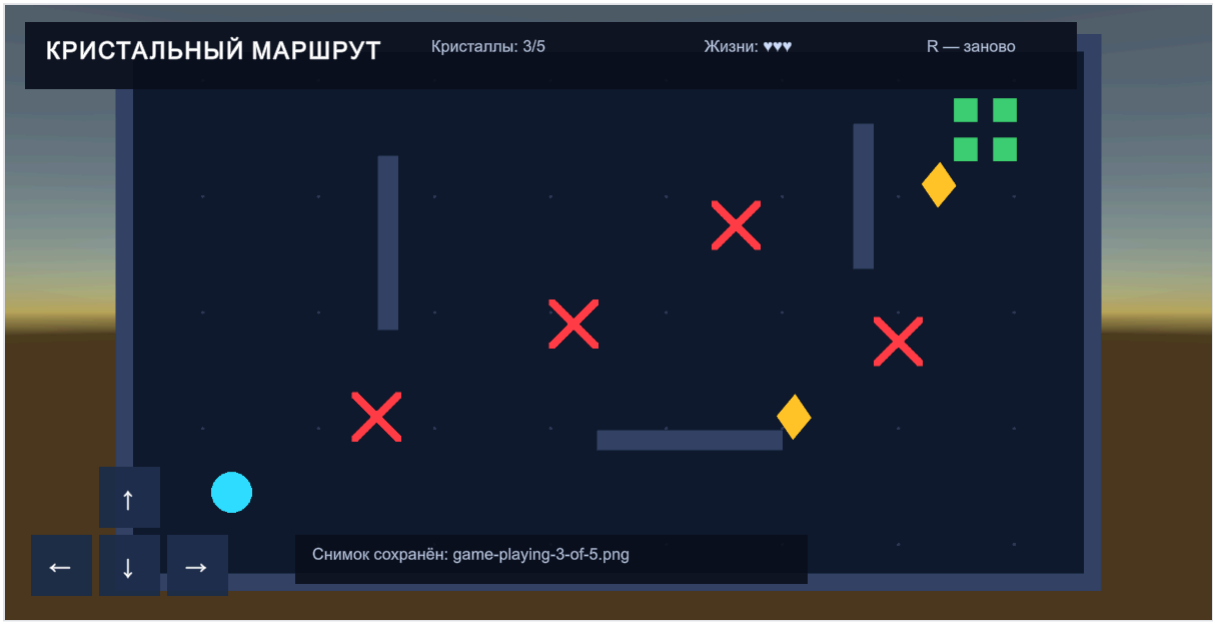
2. Проверьте опасность

Подведите игрока к красному кресту. Ожидается: игрок возвращается в левый нижний угол, число сердец уменьшается на одно. В течение одной секунды повторный урон не начисляется.

3

Если кристалл не исчез

Проверьте наличие Collider2D у игрока и кристалла, Is Trigger у кристалла и Rigidbody2D у игрока. В данном проекте компоненты создаёт GameManager; ошибка обычно означает неполный код.



Три предмета собраны, игра продолжается.

Этап 6 / Победа, поражение, повтор

Проверьте завершение

1

1. Войдите в портал раньше времени

Соберите не все кристаллы и войдите в зелёный портал. Ожидается подсказка «Сначала соберите все кристаллы». Игра продолжается.

2

2. Получите победу

Соберите пять кристаллов и войдите в портал. Ожидается панель «УРОВЕНЬ ПРОЙДЕН». Нажатия управления больше не двигают игрока.

3

3. Получите поражение

Нажмите «НАЧАТЬ ЗАНОВО». Три раза коснитесь опасности, выдерживая паузу после каждого касания. Ожидается панель «ПОПРОБУЙТЕ ЕЩЁ».

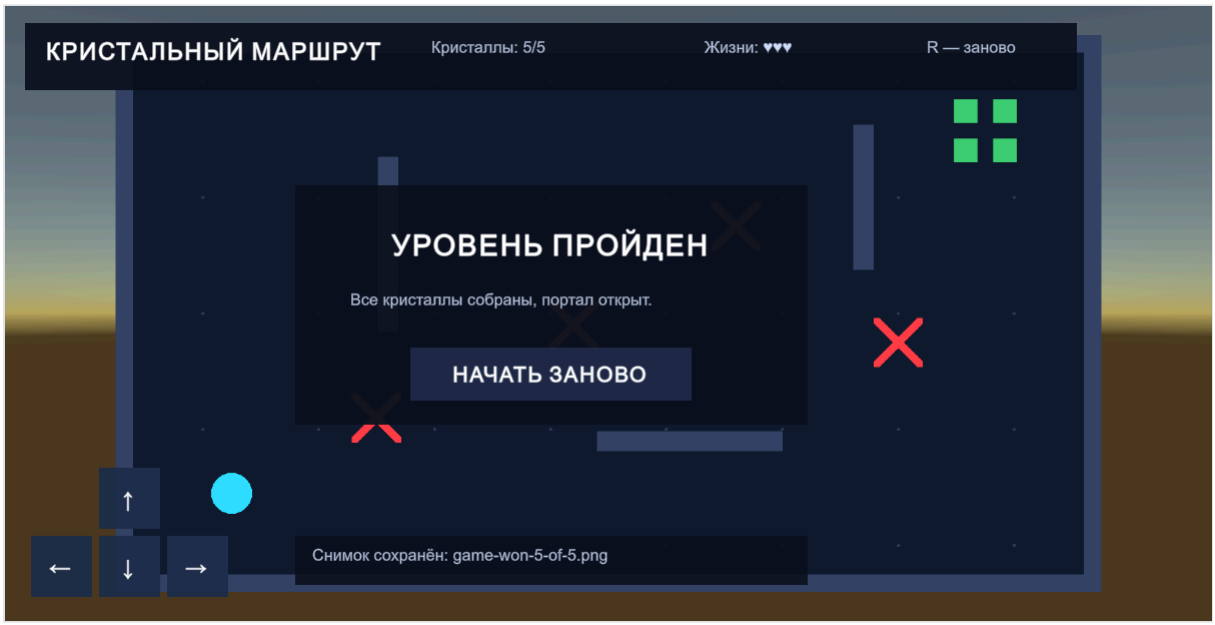
4

4. Начните заново

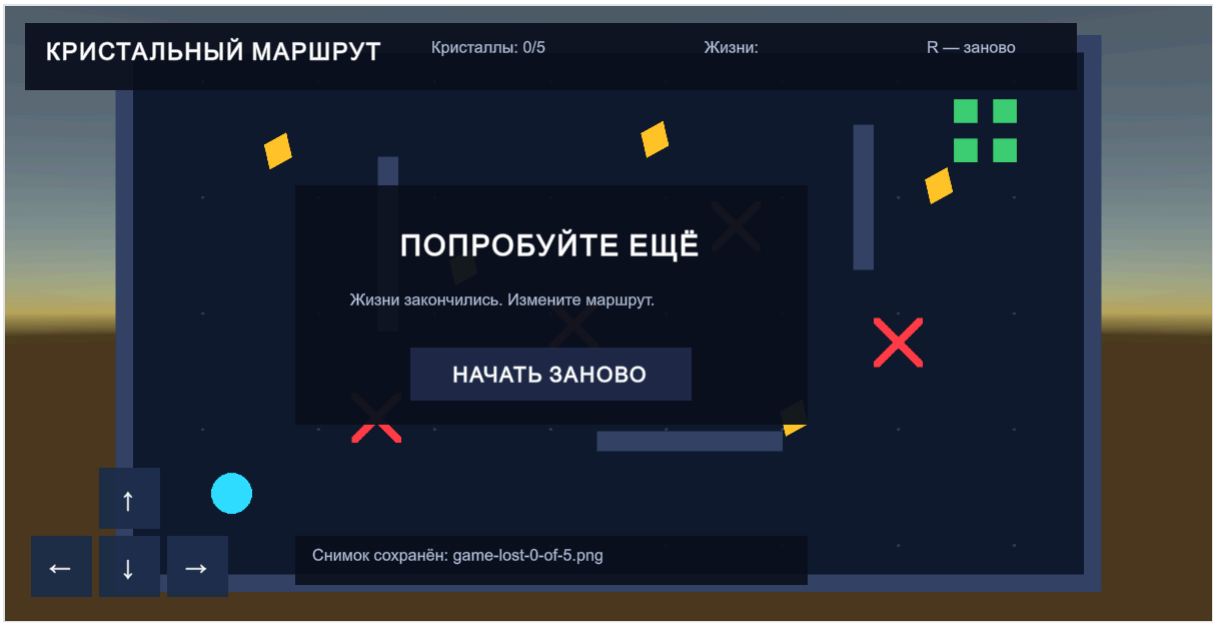
Нажмите «НАЧАТЬ ЗАНОВО». Ожидается: 0/5, три сердца, пять кристаллов снова на поле, игрок в начальной точке.

Этап 6 / Сравните изображения

Ожидаемые экраны результата



Все пять кристаллов собраны; показана кнопка повторного запуска.



Жизни закончились; показана кнопка повторного запуска.

Этапы 2–4 / Сцена строится программно

Как настраиваются визуальные элементы

1

1. Не создавайте объекты вручную

До запуска Hierarchy содержит только сцену Main. Это правильно. GameBootstrap автоматически создаёт объект CrystalQuest, а GameManager создаёт камеру, поле, стены, игрока, кристаллы, опасности и выход. Не добавляйте вручную Canvas, SpriteRenderer или Collider2D — появятся дубликаты.

2

2. Откройте настройки внешнего вида

В окне Project нажмите Assets → Scripts. Дважды щёлкните GameManager. В VS Code нажмите Ctrl+F или Cmd+F, введите BuildLevel и нажмите Enter. Ниже видны вызовы CreateSpriteObject для игрока, опасностей и выхода.

3

3. Читайте параметры слева направо

В CreateSpriteObject идут: имя; позиция new Vector2(x, y); размер new Vector2(width, height); цвет new Color(red, green, blue); форма Shape; слой отображения. Цветовые значения находятся от 0 до 1. Больший слой рисуется поверх меньшего.

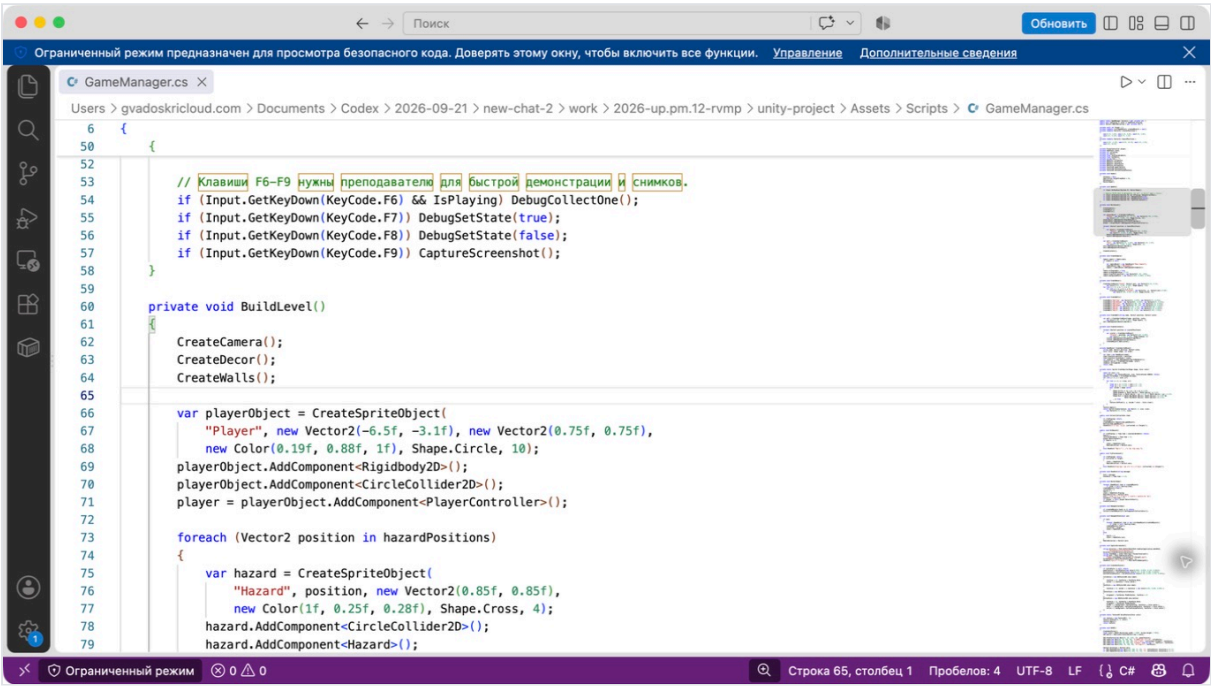
4

4. Меняйте по одному значению

Сначала измените только размер или цвет одного объекта. Нажмите Save, вернитесь в Unity, дождитесь компиляции и нажмите Play. Сравните Game View. Если результат верен, остановите Play и переходите к следующему параметру.

Этапы 2–4 / VS Code

Где находятся параметры внешнего вида



Строки внутри BuildLevel задают позицию, размер, цвет, форму и порядок отображения. В примере игрок голубой и круглый, опасность красная и имеет форму креста. После изменения сохраните файл и повторно нажмите Play.

Этап 3 / Точный маршрут по кнопкам

Расположение объектов через окно Unity

1

1. Запустите готовую основу

В Project дважды щёлкните Assets → Scenes → Main. Нажмите Play — треугольник по центру верхней панели. Нажмите вкладку Game рядом со Scene. Проверьте, что видны поле, голубой игрок, жёлтые кристаллы, красные опасности и зелёный выход.

2

2. Перейдите к размещению

Не останавливая Play, нажмите вкладку Scene. В верхней части Scene включите кнопку 2D. Если поле потерялось, в Hierarchy нажмите Player и нажмите клавишу F — камера Scene приблизится к выбранному объекту.

3

3. Найдите объект в Hierarchy

В левой панели Hierarchy прокрутите список. Нажмите Player. Для предмета нажмите один из объектов Crystal, для опасности — Hazard, для выхода — Exit. Выделенная строка становится синей, а справа Inspector показывает её параметры.

4

4. Сверьте Transform

В Inspector раскройте Transform. Position X — положение по горизонтали, Position Y — по вертикали, Z оставьте 0. Scale X и Y задают размер. Rotation X, Y и Z для этого примера оставьте 0.

Этап 3 / Scene, Move Tool и Transform

Как передвинуть объект мышью

1

5. Включите инструмент перемещения

Нажмите клавишу W или кнопку Move Tool на левой панели Scene — значок со стрелками. На выбранном объекте появятся красная горизонтальная и зелёная вертикальная стрелки.

2

6. Передвиньте по одной оси

Потяните красную стрелку только влево или вправо. Затем потяните зелёную только вверх или вниз. Не тяните центральный квадрат: так легко случайно изменить сразу обе координаты.

3

7. Введите точное значение

Для точного расположения нажмите число Position X в Inspector, нажмите Cmd+A или Ctrl+A, введите координату и нажмите Enter. Аналогично заполните Position Y. Например, для игрока X = -6.5, Y = -3.1, Z = 0.

4

8. Проверьте в Game

Нажмите вкладку Game. Объект должен находиться внутри тёмного поля и не перекрывать стену. Вернитесь в Scene, если нужно поправить координату. Пока включён Play, это только пробное размещение.

Этап 3 / Из временного положения в код

Как сохранить выбранное расположение

1

9. Запишите координаты

Оставьте объект выделенным и перепишите Position X и Position Y из Inspector в черновик. Для нескольких одинаковых объектов записывайте имя и пару координат отдельной строкой.

2

10. Остановите Play

Нажмите синюю кнопку Play вверху. Созданные объекты исчезнут из Hierarchy — это правильно: они создавались программой. Изменения Transform во время запуска также исчезнут, поэтому координаты нужно перенести в GameManager.cs.

3

11. Откройте GameManager.cs

В Project нажмите Assets → Scripts и дважды щёлкните GameManager. В VS Code нажмите Cmd+F или Ctrl+F. Для кристаллов найдите crystalPositions, для опасностей — hazardPositions, для игрока найдите строку с "Player", для выхода — строку с "Exit".

4

12. Замените одну пару и проверьте

Введите записанные X и Y внутрь new Vector2(X, Y). Десятичная часть отделяется точкой, после числа ставится f: new Vector2(-6.5f, -3.1f). Сохраните файл, вернитесь в Unity, дождитесь компиляции и снова нажмите Play.

Этап 3 / Введите и сверяйте по одному объекту

Точные координаты готового примера

1

Игрок и выход

Player: Position (-6.5, -3.1, 0), Scale (0.75, 0.75, 1). Exit: Position (6.5, 3.15, 0), Scale (1.2, 1.2, 1). Игрок должен быть в левом нижнем углу, выход — в правом верхнем.

2

Кристаллы 1–5

Crystal 1: (-5.7, 2.8). Crystal 2: (-2.5, 0.8). Crystal 3: (0.8, 3.0). Crystal 4: (3.2, -1.8). Crystal 5: (5.7, 2.2). Размер каждого: (0.62, 0.82). Z для всех равен 0.

3

Опасности 1–4

Hazard 1: (-4.0, -1.8). Hazard 2: (-0.6, -0.2). Hazard 3: (2.2, 1.5). Hazard 4: (5.0, -0.5). Размер каждой опасности: (0.85, 0.85). Z для всех равен 0.

4

Граница безопасного размещения

Для центра обычного предмета используйте X от -7.4 до 7.4 и Y от -3.9 до 3.9. Между центрами двух объектов оставляйте не менее 1 единицы. Не ставьте предмет на стену, выход или начальную точку игрока.

Этап 3 / Фон, рамка и препятствия

Точные координаты стен и поля

1

Поле и камера

Field: Position (0, 0, 0), Scale (16.4, 9.2, 1), Order in Layer -10. Main Camera: Position (0, 0, -10), Projection Orthographic, Size 5.3. Эти значения позволяют видеть игровое поле целиком.

2

Внешняя рамка

WallTop: Position (0, 4.65), Scale (17, 0.3). WallBottom: (0, -4.65), Scale (17, 0.3). WallLeft: (-8.35, 0), Scale (0.3, 9.6). WallRight: (8.35, 0), Scale (0.3, 9.6).

3

Внутренние стены

WallA: Position (-3.8, 1.2), Scale (0.35, 3.0). WallB: Position (1.4, -2.2), Scale (3.2, 0.35). WallC: Position (4.4, 2.0), Scale (0.35, 2.5). Rotation везде (0, 0, 0).

4

Как изменить стену

Во время Play выберите стену в Hierarchy, нажмите W и найдите положение. Затем остановите игру. В GameManager.cs нажмите Cmd+F или Ctrl+F, найдите CreateWalls и замените соответствующие Vector2 позиции и размера. Не удаляйте BoxCollider2D.

Этап 3 / Перед программированием правил

Проверка расположения в Unity

1

13. Проверьте обзор

Нажмите Play → Game. Виден весь прямоугольник поля, четыре внешние стены и три внутренние стены. Ни один объект не обрезан краем Game View.

2

14. Проверьте свободные проходы

Стрелками или WASD проведите игрока от начальной точки к каждому кристаллу. Если пройти нельзя, остановите Play и передвиньте ближайшую стену или предмет минимум на 0.5 по X либо Y.

3

15. Проверьте столкновения

Игрок не должен проходить через стены. Кристалл должен исчезать при касании. Опасность должна уменьшать жизнь. Exit должен реагировать только при входе игрока. При неверном результате откройте Inspector объекта во время Play и проверьте Collider2D.

4

16. Зафиксируйте результат

Остановите Play. Нажмите File → Save. Снова запустите игру и повторите короткий маршрут. Сделайте снимок Game View только после повторного запуска — так подтверждается, что координаты сохранены в коде.

Подготовьте правило своего варианта

1

Что сохраняется

Движение и физика игрока, камера, стены, панель результата и повтор остаются. Изменяются назначение предметов, состояние, контакты, интерфейс и условие завершения. Выпишите из карточки численные цели и реакцию на ошибку.

2

Добавьте учебные модели

В Assets/Scripts создайте MonoBehaviour Script с именем MechanicModels. Полностью удалите шаблон MonoBehaviour; это обычный C#-класс данных, который не прикрепляют к GameObject. Введите следующие логические блоки в одном файле; последнее объявление закрывает общий класс. Модели отделяют правила от изображения и позволяют проверить состояние.

3

Важное ограничение

Выберите одну группу своего варианта. Учебная модель — пример устройства состояния, а не готовая программа варианта. Для правил со сбросом собранных объектов восстановите их визуально, для повреждений меняйте жизни через HitHazard, для замедления добавляйте явное изменение скорости. Не используйте модель без подключения объектов, интерфейса и повторного запуска.

4

Место для нового индикатора

В OnGUI после верхних GUI.Label добавьте строку собственного правила ниже шапки: GUI.Label(new Rect(240, 90, 900, 45), текстСостояния, textStyle);. Вместо текстСостояния подставьте выражение своей механики, например «Следующий: » + sequence.Expected. Rect здесь использует те же координаты 1280×720, что и остальной интерфейс. Не добавляйте новый текст на высоте 35–55: там уже находятся название, прогресс и жизни. Проверьте читаемость в Game 16:9 и при уменьшении окна. Для комплекта и переключателей также замените верхнюю надпись «Кристаллы: collected/Target» показателем состава или выполненных условий: условие kit.Complete либо access.All не связано с пятью кристаллами примера. Не оставляйте на экране противоречащие результату показатели.

ПМ.12 / Создание, объяснение и проверка

Модели правил / Общий класс

1

Файл и место

Assets/Scripts/MechanicModels.cs. Начните пустой файл. Класс становится компилируемым после последнего блока.

2

Состояние и операции Общий класс

```
// Учебные модели правил. Unity-объекты и интерфейс подключаются отдельно.
public static class MechanicModels
{
```

3

Проверка смысла

Операция возвращает bool: true означает принятое действие, false — отказ. Счётчик меняется только в принятой ветви. Reset возвращает всё состояние модели к исходному; вызовите его при каждом RestartGame. Для Access повторная активация одного индекса возвращает false.

ПМ.12 / Создание, объяснение и проверка

Модели правил / Selection

1

Файл и место

Assets/Scripts/MechanicModels.cs. Продолжите сразу после предыдущей модели. Класс становится компилируемым после последнего блока.

2

Состояние и операции Selection

```
public sealed class Selection
{
    public int Progress { get; private set; }
    public int Errors { get; private set; }
    public bool Take(bool correct)
    {
        if (!correct) { Errors++; return false; }
        Progress++; return true;
    }
    public void Reset() { Progress = 0; Errors = 0; }
}
```

3

Проверка смысла

Операция возвращает bool: true означает принятое действие, false — отказ. Счётчик меняется только в принятой ветви. Reset возвращает всё состояние модели к исходному; вызовите его при каждом RestartGame. Для Access повторная активация одного индекса возвращает false.

Модели правил / Sequence

1

Файл и место

Assets/Scripts/MechanicModels.cs. Продолжите сразу после предыдущей модели. Класс становится компилируемым после последнего блока.

2

Состояние и операции Sequence

```
public sealed class Sequence
{
    public int Expected { get; private set; } = 1;
    public int Errors { get; private set; }
    public bool Take(int index, bool resetOnError)
    {
        if (index != Expected)
        {
            Errors++;
            if (resetOnError) Expected = 1;
            return false;
        }
        Expected++; return true;
    }
    public void Reset() { Expected = 1; Errors = 0; }
}
```

3

Проверка смысла

Операция возвращает bool: true означает принятое действие, false — отказ. Счётчик меняется только в принятой ветви. Reset возвращает всё состояние модели к исходному; вызовите его при каждом RestartGame. Для Access повторная активация одного индекса возвращает false.

Модели правил / Delivery

1

Файл и место

Assets/Scripts/MechanicModels.cs. Продолжите сразу после предыдущей модели. Класс становится компилируемым после последнего блока.

2

Состояние и операции Delivery

```
public sealed class Delivery
{
    public bool HasCargo { get; private set; }
    public int Delivered { get; private set; }
    public bool Pickup()
    {
        if (HasCargo) return false;
        HasCargo = true; return true;
    }
    public bool Unload()
    {
        if (!HasCargo) return false;
        HasCargo = false; Delivered++; return true;
    }
    public bool Drop()
    {
        if (!HasCargo) return false;
        HasCargo = false; return true;
    }
    public void Reset() { HasCargo = false; Delivered = 0; }
}
```

3

Проверка смысла

Операция возвращает bool: true означает принятое действие, false — отказ. Счётчик меняется только в принятой ветви. Reset возвращает всё состояние модели к исходному; вызовите его при каждом RestartGame. Для Access повторная активация одного индекса возвращает false.

Модели правил / Capacity

1

Файл и место

Assets/Scripts/MechanicModels.cs. Продолжите сразу после предыдущей модели. Класс становится компилируемым после последнего блока.

2

Состояние и операции Capacity

```
public sealed class Capacity
{
    public int Carried { get; private set; }
    public int Delivered { get; private set; }
    public bool Pickup(int capacity)
    {
        if (Carried >= capacity) return false;
        Carried++; return true;
    }
    public int Unload()
    {
        int amount = Carried; Delivered += amount; Carried = 0;
        return amount;
    }
    public void Reset() { Carried = 0; Delivered = 0; }
}
```

3

Проверка смысла

Операция возвращает bool: true означает принятое действие, false — отказ. Счётчик меняется только в принятой ветви. Reset возвращает всё состояние модели к исходному; вызовите его при каждом RestartGame. Для Access повторная активация одного индекса возвращает false.

Модели правил / Kit

1

Файл и место

Assets/Scripts/MechanicModels.cs. Продолжите сразу после предыдущей модели. Класс становится компилируемым после последнего блока.

2

Состояние и операции Kit

```
    public sealed class Kit
    {
        private readonly int[] counts = new int[3];
        public int Count(int type) => counts[type];
        public bool Take(int type, int required)
        {
            if (type < 0 || type >= counts.Length || counts[type] >= required)
                return false;
            counts[type]++; return true;
        }
        public bool Complete(int a, int b, int c)
            => counts[0] >= a && counts[1] >= b && counts[2] >= c;
        public void Reset() { for (int i = 0; i < counts.Length; i++) counts[i]
= 0; }
    }
```

3

Проверка смысла

Операция возвращает bool: true означает принятое действие, false — отказ. Счётчик меняется только в принятой ветви. Reset возвращает всё состояние модели к исходному; вызовите его при каждом RestartGame. Для Access повторная активация одного индекса возвращает false.

Модели правил / Access

1

Файл и место

Assets/Scripts/MechanicModels.cs. Продолжите сразу после предыдущей модели. Класс становится компилируемым после последнего блока.

2

Состояние и операции Access

```
    public sealed class Access
    {
        private readonly bool[] flags = new bool[3];
        public bool Activate(int index)
        {
            if (index < 0 || index >= flags.Length || flags[index]) return
false;
            flags[index] = true; return true;
        }
        public bool Enabled(int index) => flags[index];
        public bool All(int required)
        {
            for (int i = 0; i < required; i++) if (!flags[i]) return false;
            return true;
        }
        public void Reset() { for (int i = 0; i < flags.Length; i++) flags[i] =
false; }
    }
}
```

3

Проверка смысла

Операция возвращает bool: true означает принятое действие, false — отказ. Счётчик меняется только в принятой ветви. Reset возвращает всё состояние модели к исходному; вызовите его при каждом RestartGame. Для Access повторная активация одного индекса возвращает false.

ПМ.12 / Создание, объяснение и проверка

Свяжите игровые предметы с данными

1

Как добавить признаки

В Assets/Scripts/Collectible.cs внутри класса перед private Vector3 startScale добавьте public int itemIndex = 1; public int itemType = 0;. Это номер и тип предмета; они не являются координатами или цветом. В Awake значения не меняйте.

2

Где назначать значения

В GameManager.CreateCrystals замените foreach по позициям циклом for (int i = 0; i < crystalPositions.Length; i++). В начале тела задайте Vector2 position = crystalPositions[i];. Строку crystal.AddComponent<Collectible>(); замените на var item = crystal.AddComponent<Collectible>(); item.itemIndex = i + 1; item.itemType = i % 3;. Остальное тело оставьте. Для своей области задайте номера и типы по её правилам, а не случайно.

3

Визуальное различие

Чтобы номер был виден игроку, в GameManager.OnGUI после надписей счётчиков переберите FindObjectsByType<Collectible> (FindObjectsSortMode.None). Для каждого предмета получите Camera.main.WorldToScreenPoint(item.transform.position), разделите экранные X и (Screen.height - Y) на scale и нарисуйте GUI.Label в Rect у предмета с item.itemIndex.ToString(). Для типов используйте различимые формы и понятные подписи. Цвет не должен быть единственным признаком.

4

Проверка соединения

После Save All запустите Main. Выделите Crystal в Hierarchy во время Play: Inspector показывает Item Index и Item Type. Проверьте все созданные значения. Исправляйте их в CreateCrystals: ручная правка во время Play не сохраняется. Пока Collect не изменён, обычный сбор должен продолжать работать.

ПМ.12 / Создание, объяснение и проверка

Выборочный сбор / варианты 01–05

1

Добавьте состояние

В GameManager внутри класса рядом с collected добавьте private readonly MechanicModels.Selection selection = new();. В RestartGame после collected = 0 добавьте selection.Reset();. Сначала сохраните и убедитесь, что обычная игра компилируется.

2

Место изменения

В Collect замените collected++ приведённым блоком. Удаление принятого предмета из createdObjects и Destroy оставьте ниже. Непринятый предмет остаётся: не выполняйте Destroy в ветви отказа. itemType == 0 здесь обозначает правильный тип; назначьте типы по карточке.

3

Фрагмент внутри Collect после проверки IsPlaying

```
if (!selection.Take(item.itemType == 0))
{
    HitHazard();
    return;
}
collected = selection.Progress;
```

4

Почему это работает

Selection хранит правильные действия отдельно от ошибок. false сохраняет неподходящий предмет; true разрешает увеличить collected и удалить принятый объект.

5

Интерфейс, завершение и связи

Покажите в OnGUI collected/Target и жизни. Для заряда, запаса времени и ошибок применяйте отдельные обработчики из раздела о причинах поражения. Отдельные опасности по-прежнему вызывают HitHazard.

6

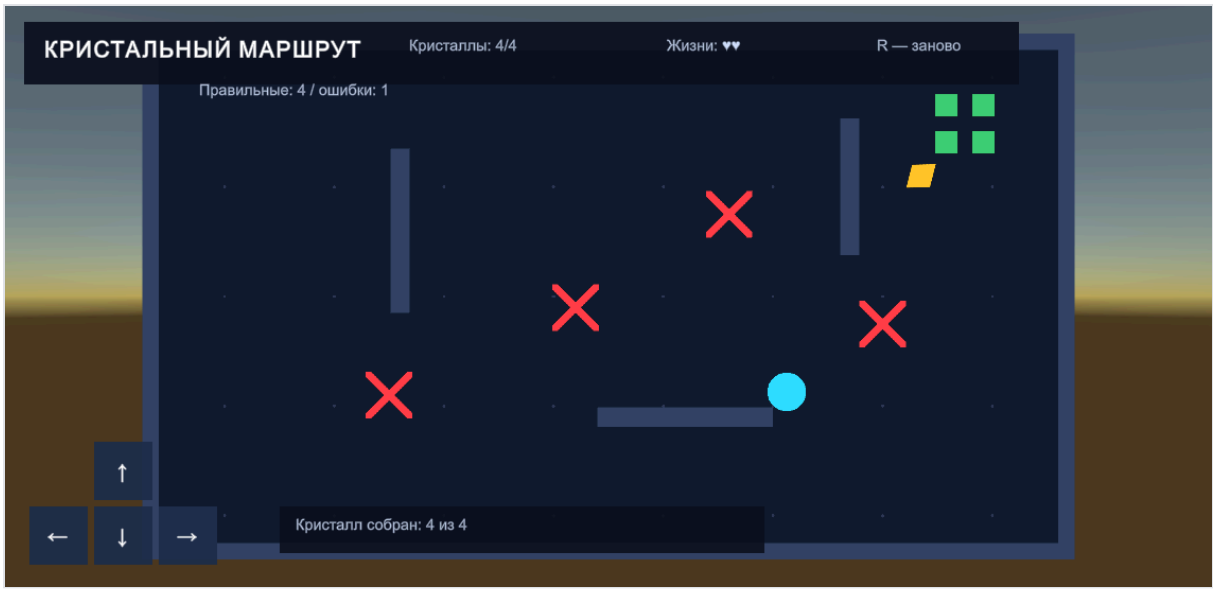
Проверка результата

Один правильный предмет увеличивает прогресс на один. Неправильный не увеличивает прогресс и вызывает предусмотренный штраф. До цели портал отказывает; после цели принимает. При повторе прогресс и штраф сброшены.

7

Регрессия

Повторите общую проверку: движение, стену, опасность, ранний финиш, победу, поражение и новый запуск. Затем выполните обязательную проверку своей карточки.



Четыре правильных предмета приняты; ошибочный тип не засчитан, одна жизнь потеряна. Число целей задаётся правилом.

ПМ.12 / Создание, объяснение и проверка

Сбор по порядку / варианты 06–10

1

Добавьте состояние

В GameManager внутри класса рядом с collected добавьте private readonly MechanicModels.Sequence sequence = new();. В RestartGame после collected = 0 добавьте sequence.Reset();. Сначала сохраните и убедитесь, что обычная игра компилируется.

2

Место изменения

В Collect замените collected++ блоком. false означает: неправильный номер не меняет ожидаемый номер, но увеличивает ошибки. В вариантах со сбросом используйте true, обнулите collected и восстановите ранее исчезнувшие предметы: очистите createdObjects и вызовите CreateCrystals. Не меняйте порядок массива без согласования номеров.

3

Фрагмент внутри Collect после проверки IsPlaying

```
if (!sequence.Take(item.itemIndex, false))
{
    HitHazard();
    return;
}
collected = sequence.Expected - 1;
```

4

Почему это работает

Expected хранит номер следующего разрешённого действия. Принятый номер продвигает его; отказ сохраняет предмет. Сброс порядка является отдельным правилом варианта.

5

Интерфейс, завершение и связи

В OnGUI покажите «Следующий: » + sequence.Expected. Финиш разрешайте при collected == Target. Для последовательности команд робота действие должно менять состояние команды, а не только счётчик: свяжите itemIndex с соответствующим действием своей карточки.

6

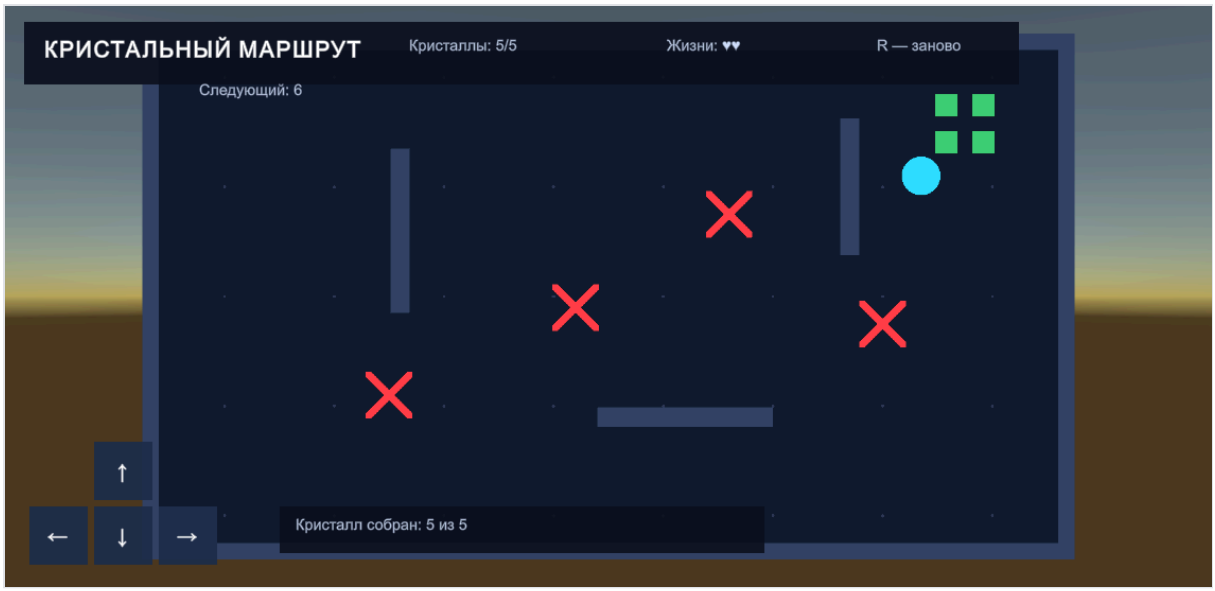
Проверка результата

Проверьте номера 1, затем 3, затем 2. После ошибочного 3 ожидаемый номер должен соответствовать правилу карточки. Повтор возвращает Expected к 1 и восстанавливает предметы.

7

Регрессия

Повторите общую проверку: движение, стену, опасность, ранний финиш, победу, поражение и новый запуск. Затем выполните обязательную проверку своей карточки.



Пять предметов приняты в заданном порядке; следующий индекс — 6. Неверный контакт не удалил предмет и не продвинул ожидаемый номер.

ПМ.12 / Создание, объяснение и проверка

Подбор и доставка / варианты 11–15

1

Добавьте состояние

В GameManager внутри класса рядом с collected добавьте private readonly MechanicModels.Delivery delivery = new();. В RestartGame после collected = 0 добавьте delivery.Reset();. Сначала сохраните и убедитесь, что обычная игра компилируется.

2

Место изменения

В Collect уберите collected++; вместо него используйте Pickup. Принятый предмет удаляйте, но collected при подборе не растёт. В GameManager добавьте public void DeliverCargo() с блоком доставки ниже. В ExitZone.OnTriggerEnter2D замените вызов TryFinishLevel на DeliverCargo, если Exit используется как амбар/склад. В DeliverCargo после успешной разгрузки вызовите TryFinishLevel: последняя доставка завершает игру.

3

Фрагмент внутри Collect после проверки IsPlaying

```
if (!delivery.Pickup())
{
    ShowHint("Сначала доставьте текущий груз");
    return;
}
```

4

Почему это работает

HasCargo ограничивает подбор одним предметом. Delivered растёт только при успешной разгрузке; повторный вход с пустым грузом ничего не добавляет.

5

Интерфейс, завершение и связи

Блок доставки: if (!IsPlaying || !delivery.Unload()) return; collected = delivery.Delivered; ShowHint("Груз доставлен"); TryFinishLevel();. Выведите HasCargo и Delivered в OnGUI. При наличии отдельной финальной зоны создайте второй скрипт триггера DeliveryZone, скопировав проверку игрока из ExitZone; только он вызывает DeliverCargo, а финальный Exit сохраняет TryFinishLevel.

6

Вариант 12. Вход в ящик без письма

В GameManager добавьте поле для пустых входов, например missedDeliveries. В RestartGame обнуляйте его вместе с delivery.Reset(). Оставьте вызов DeliverCargo в ExitZone.OnTriggerEnter2D. В BuildLevel уберите создание опасных зон, чтобы единственным условием поражения остались пустые входы. Покажите на экране число доставок, число пустых входов и есть ли письмо у игрока. Замените в TryFinishLevel подсказку про кристаллы на сообщение о доставленных письмах.

7

Вариант 12. Что делает обработчик DeliverCargo

Обрабатывайте вход только в состоянии Playing. Если delivery.Unload() вернул false, прибавьте один к missedDeliveries, покажите новый счётчик и завершите обработку входа. При третьем пустом входе перед выходом из метода задайте состояние Lost и обнулите MobileDirection. Если разгрузка удалась, перенесите delivery.Delivered в collected и вызовите TryFinishLevel. Победу объявляет пятая доставка. Для проверки постойте внутри ящика после пустого входа. За это время счётчик не должен увеличиться повторно.

8

Вариант 12. Проверьте счётчик попыток

Используйте портал из примера как ящик. С пустыми руками войдите в него и постойте внутри: счётчик должен увеличиться на 1. Выйдите и войдите ещё два раза: при третьем пустом входе покажите поражение. Нажмите «НАЧАТЬ ЗАНОВО»: счётчики и груз должны сброситься, все пять писем снова появятся. Подберите письмо, попробуйте взять второе и проверьте, что второе осталось на поле. Доставьте пять писем и проверьте победу.

9

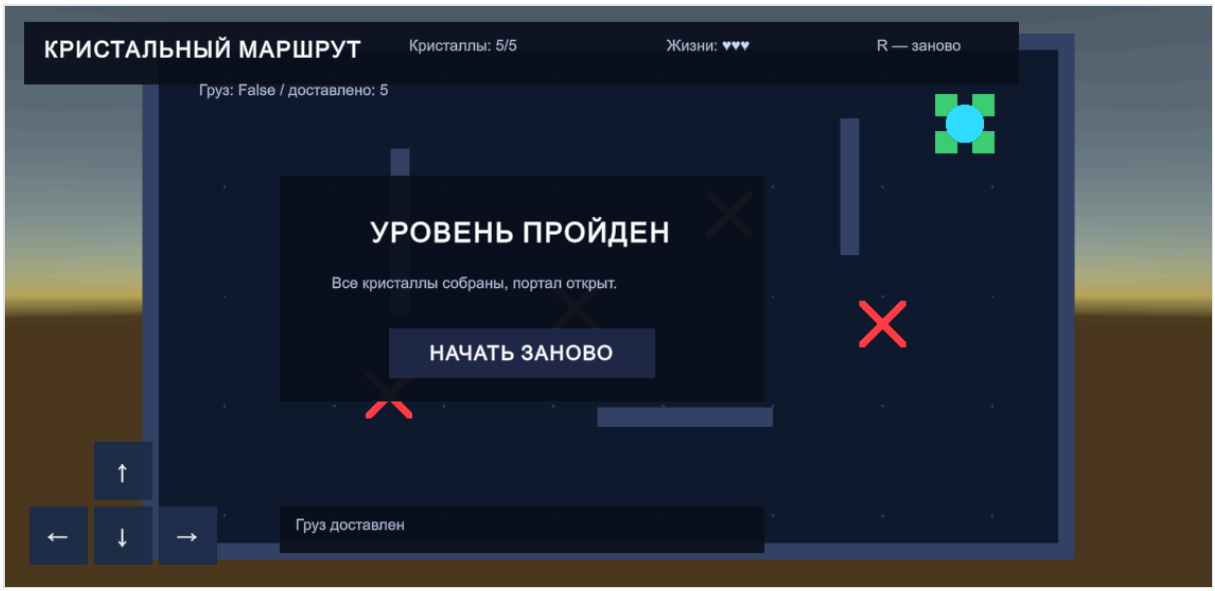
Проверка результата

Вход в склад без груза ничего не засчитывает. Один подбор разрешён, второй до разгрузки отвергается. Вход с грузом прибавляет ровно один Delivered и снимает груз. Повтор обнуляет HasCargo и Delivered.

10

Регрессия

Проверьте движение, стену, раннее завершение, победу, поражение и новый запуск. Используйте ошибочное действие или опасность из своей карточки. В варианте 12 проверьте пустые входы в ящик.



После пяти фактических доставок груз пуст, счётчик доставки равен пяти, показана победа. Подбор сам по себе не увеличивает доставленный результат.

ПМ.12 / Создание, объяснение и проверка

Ограниченная вместимость / варианты 16–20

1

Добавьте состояние

В GameManager внутри класса рядом с collected добавьте private readonly MechanicModels.Capacity capacityRule = new();. В RestartGame после collected = 0 добавьте capacityRule.Reset();. Сначала сохраните и убедитесь, что обычная игра компилируется.

2

Место изменения

В Collect уберите collected++, вставьте проверку Pickup, затем удаляйте только принятый предмет. Значение 3 замените вместимостью карточки. Добавьте public void UnloadCargo(): if (!IsPlaying) return; int amount = capacityRule.Unload(); if (amount == 0) return; collected = capacityRule.Delivered; ShowHint("Разгружено: " + amount); TryFinishLevel();. Триггер склада вызывает UnloadCargo; финальный выход остаётся отдельным при необходимости.

3

Фрагмент внутри Collect после проверки IsPlaying

```
if (!capacityRule.Pickup(3))
{
    ShowHint("Нет свободного места");
    return;
}
```

4

Почему это работает

Carried ограничивает текущую загрузку. Unload переносит её в Delivered и обнуляет груз; общее число доставок может превышать вместимость одного рейса.

5

Интерфейс, завершение и связи

Выведите Carried, вместимость и Delivered. В TryFinishLevel используйте условие delivered цели карточки. Если цель превышает пять исходных предметов, расширьте crystalPositions до нужного количества; иначе победа недостижима. Непринятый предмет должен оставаться доступным после разгрузки.

6

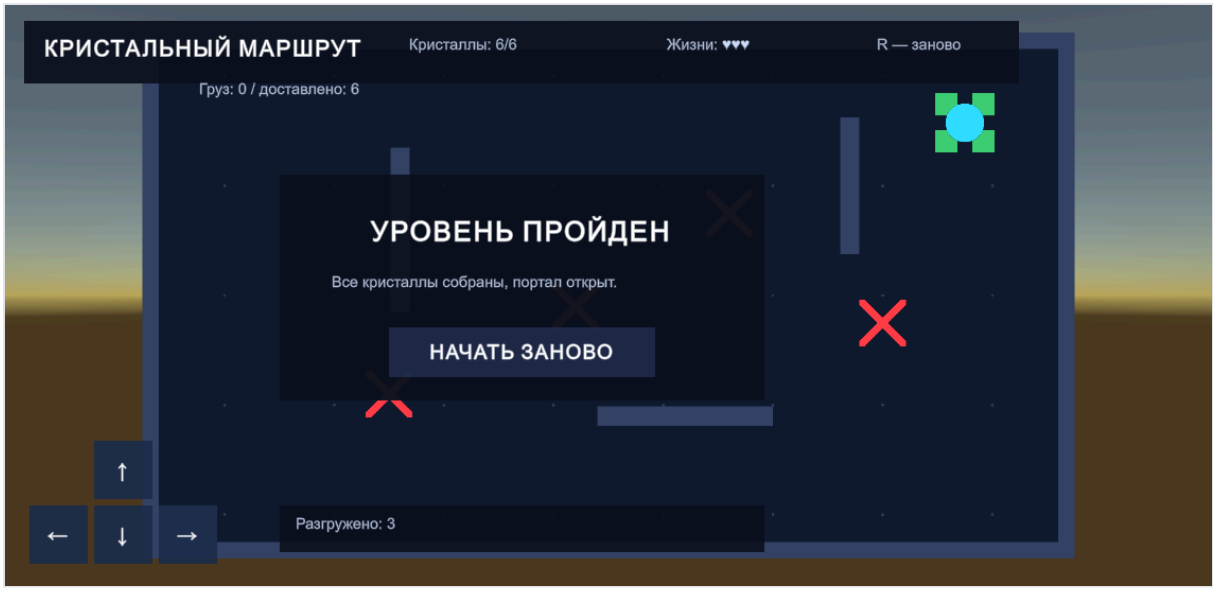
Проверка результата

Возьмите ровно capacity предметов, затем попытайтесь взять ещё один. Carried не превышает capacity. Пустая разгрузка не меняет Delivered; непустая переносит весь груз и обнуляет Carried. Повтор возвращает оба счётчика к нулю.

7

Регрессия

Повторите общую проверку: движение, стену, опасность, ранний финиш, победу, поражение и новый запуск. Затем выполните обязательную проверку своей карточки.



Два рейса с вместимостью три дали шесть доставленных предметов; груз пуст, игра завершена. Четвёртый предмет до разгрузки отклонялся.

ПМ.12 / Создание, объяснение и проверка

Комплектование / варианты 21–25

1

Добавьте состояние

В GameManager внутри класса рядом с collected добавьте private readonly MechanicModels.Kit kit = new();. В RestartGame после collected = 0 добавьте kit.Reset();. Сначала сохраните и убедитесь, что обычная игра компилируется.

2

Место изменения

Замените collected++ в Collect этим блоком. Пример требует два предмета типа 0 и по одному типов 1 и 2. Для карточки назначьте нужные количества и набор типов. В TryFinishLevel замените collected == Target на kit.Complete(2, 1, 1). Для карточки из двух типов третий требуемый счётчик равен 0.

3

Фрагмент внутри Collect после проверки IsPlaying

```
int required = item.itemType == 0 ? 2 : 1;
if (!kit.Take(item.itemType, required))
{
    ShowHint("Этот тип уже набран");
    return;
}
collected++;
```

4

Почему это работает

Три счётчика типов проверяют состав, а не сумму. Лишний предмет не заполняет другой тип; Complete сравнивает каждый счётчик с собственной целью.

5

Интерфейс, завершение и связи

Покажите Count(0), Count(1), Count(2) и соответствующие цели. CreateCrystals должен создавать достаточное количество каждого типа. При полном типе отказ оставляет лишний предмет на поле; если карточка требует штраф, добавьте его явно в ветвь false. Для доставки готового комплекта завершение проверяется при входе в анализатор/лагерь.

6

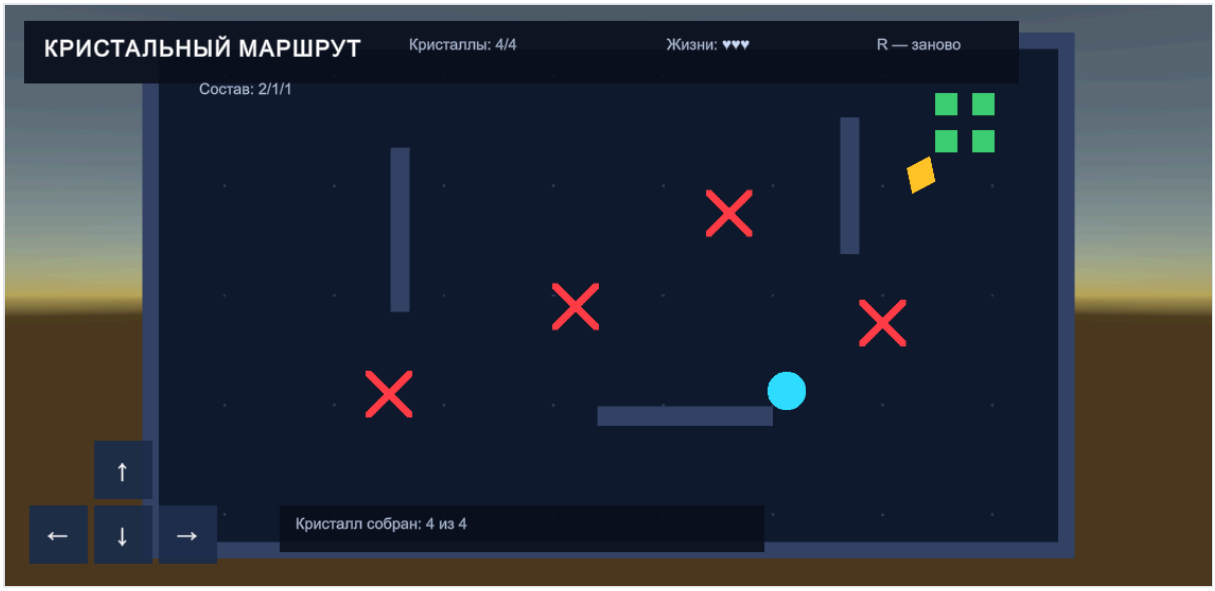
Проверка результата

Неполный комплект не разрешает финиш. Повтор одного уже набранного типа не заменяет другой тип. Проверьте разные порядки правильного сбора и лишний предмет. Reset обнуляет все три счётчика.

7

Регрессия

Повторите общую проверку: движение, стену, опасность, ранний финиш, победу, поражение и новый запуск. Затем выполните обязательную проверку своей карточки.



Учебный набор 2/1/1 полон; избыточный предмет остался на поле. Для своего варианта используйте его состав.

ПМ.12 / Создание, объяснение и проверка

Ключи, переключатели и доступ / варианты 26–30

1

Добавьте состояние

В GameManager внутри класса рядом с collected добавьте private readonly MechanicModels.Access access = new();. В RestartGame после collected = 0 добавьте access.Reset();. Сначала сохраните и убедитесь, что обычная игра компилируется.

2

Место изменения

В Collect замените collected++ блоком. Индексы примера 1–3 соответствуют трём условиям. Для двух условий достаточно индексов 1–2. В TryFinishLevel замените collected == Target на access.All(3) или All(2) по карточке. Для ключа и отдельного замка факт получения ключа должен лишь разрешать обработчик замка, а не сразу открывать все ворота.

3

Фрагмент внутри Collect после проверки IsPlaying

```
if (!access.Activate(item.itemIndex - 1))
{
    ShowHint("Условие уже выполнено");
    return;
}
collected++;
```

4

Почему это работает

Массив флагов хранит независимые условия доступа. Activate отклоняет повтор; All проверяет весь нужный набор. Коллайдер стены подключается к этому состоянию отдельно.

5

Интерфейс, завершение и связи

В CreateWall сохраните возвращённый объект нужной стены в поле GameObject gate; после активации нужного условия вызовите gate.GetComponent<Collider2D>().enabled = false; gate.GetComponent<SpriteRenderer>().color = Color.green;. Для нескольких стен храните отдельные ссылки и связывайте каждую со своим индексом. В RestartGame восстановите enabled = true и исходный цвет. Для моста меняется доступность участка пути, а не только его окраска.

6

Проверка результата

Повтор одного переключателя не заменяет второй. Проверьте дверь до активации, после одного условия и после всех. Проверьте независимость двух стен и восстановление препятствий при RestartGame.

7

Регрессия

Повторите общую проверку: движение, стену, опасность, ранний финиш, победу, поражение и новый запуск. Затем выполните обязательную проверку своей карточки.



Три независимых условия активированы; выбранная стена зелёная, Collider2D отключён. После повторного запуска коллайдер и исходный цвет восстановлены.

ПМ.12 / Проверка реализации

Сброс последовательности и отдельный счётчик ошибок

1

Восстановите данные и объекты

Для варианта 06 используйте sequence.Take(item.itemIndex, true). В ветви false присвойте collected = 0; затем foreach (GameObject remaining in createdObjects) if (remaining != null) Destroy(remaining); createdObjects.Clear(); CreateCrystals();. После этого return. Destroy удаляет старые объекты в конце кадра; не запускайте второй CreateCrystals в этом же контакте.

2

Как получить поражение от ошибок

Если карточка задаёт предел ошибок, а не повреждений, в ветви отказа сравните sequence.Errors с пределом. При достижении предела задайте state = GameState.Lost; MobileDirection = Vector2.zero;. При меньшем количестве покажите ShowHint. Не вызывайте HitHazard, если ошибка должна только сбрасывать порядок. В RestartGame sequence.Reset уже обнуляет ошибки.

3

Проверка

Соберите 1, ошибочно коснитесь 3: ожидаемый снова 1, прогресс 0, первый предмет восстановлен. Повторите до предела ошибок: наступает Lost. После нового запуска все номера доступны и Errors равен 0. В вариантах 08–10 сохраните другой предписанный эффект ошибки.

ПМ.12 / Проверка реализации

Как добавить зону разгрузки и повторяющиеся доставки

1

Новый обработчик

В Assets/Scripts создайте DeliveryZone. Скопируйте весь код ExitZone, переименуйте класс в DeliveryZone; в OnTriggerEnter2D замените GameManager.Instance.TryFinishLevel(); на GameManager.Instance.DeliverCargo(); или UnloadCargo() для вместимости. Выберите только один метод, который уже добавлен в менеджер.

2

Создание зоны в BuildLevel

Перед CreateCrystals добавьте var depot = CreateSpriteObject("Depot", new Vector2(-6f, 3f), new Vector2(1.2f, 1.2f), Color.blue, Shape.Exit, 2); depot.AddComponent<BoxCollider2D>(); depot.AddComponent<DeliveryZone>();. Awake обработчика включает Is Trigger. При необходимости измените координаты, чтобы зона не пересекала стену или предмет.

3

Отделите разгрузку от финиша

Если у игры отдельный портал, метод разгрузки не вызывает TryFinishLevel: он лишь обновляет delivered. ExitZone портала сохраняет вызов TryFinishLevel. Если склад одновременно финиш, вызов TryFinishLevel остаётся в разгрузке. Решение зависит от карточки, а не названия зоны.

4

Повторные рейсы

Для цели 6, 8, 9 или 10 предметов дополните crystalPositions соответствующим количеством доступных координат либо реализуйте явно описанное возобновление источника после успешной разгрузки. Не засчитывайте один и тот же груз повторно. Количество и размещение проверьте в начале игры.

5

Проверка

Склад без груза не меняет прогресс. Полный рейс переносит груз ровно один раз. Повторный вход без нового подбора ничего не даёт. Цель должна быть достижима количеством доступных предметов.

ПМ.12 / Проверка реализации

Связь ключа, двери и восстановление препятствий

1

Храните ссылки на стены

В GameManager рядом с player добавьте private GameObject gateA; private GameObject gateB;. В CreateWalls для выбранных стен замените CreateWall(...) на присваивание gateA = CreateSpriteObject(...) и gateA.AddComponent<BoxCollider2D>();; аналогично gateB. Имя, позицию, размер, цвет и Shape.Square возьмите из исходного вызова стены.

2

Открытие по условию

После успешного access.Activate отключайте Collider2D только нужной стены. Для двери от двух разных рубильников условие access.All(2). Для двух независимых терминалов используйте access.Enabled(0) для gateA и Enabled(1) для gateB. Если нужен контакт ключа с замком, добавьте отдельный скрипт замка по образцу ExitZone: он проверяет нужный флаг перед отключением стены.

3

Полный сброс

В RestartGame после access.Reset вызовите для каждой существующей стены GetComponent<Collider2D>().enabled = true; и восстановите исходный цвет SpriteRenderer. Флаг без физического сброса оставляет проходимую стену после нового запуска.

4

Проверка

До условия стена останавливает игрока. После нужного ключа или терминала соответствующий проход доступен. Другой терминал не открывает несвязанную стену. После повтора обе стены снова преграждают путь.

Заряд, запас времени и ошибки как разные причины поражения

1

Где хранить запас

Для карточек 02, 04, 17, 18 и 20 в GameManager рядом с hearts добавьте private int reserve = 3;. Начальный запас можно выбрать по условиям своей игры; здесь три — пример для проверки границы. В RestartGame присвойте reserve = 3;. В OnGUI добавьте подпись с назначением показателя: заряд, запас времени, кислород, энергия или батарея.

2

Уменьшение и граница

В обработчике предписанного неверного контакта вместо HitHazard уменьшайте reserve: reserve = Mathf.Max(0, reserve - 1);. Затем if (reserve == 0) { state = GameState.Lost; MobileDirection = Vector2.zero; } else ShowHint("Запас уменьшен");. Обновляйте подпись показателя. Не используйте время учебной практики: это только данные игровой модели.

3

Как выбрать событие

В 02 повреждённый модуль уменьшает заряд, в 04 лишний препарат уменьшает запас на один. Для кислорода, энергии и батареи, если событие расхода не задано, задайте явное правило в описании собственной игры, например расход при контакте с опасностью. Если выбираете непрерывный расход, добавьте lastReserveTick и уменьшайте не в каждом кадре, а при Time.time >= lastReserveTick + interval; сбрасывайте lastReserveTick при повторе.

4

Ошибки вместо жизней

В вариантах 03 и 05 неправильно выбранный тип вызывает selection.Take(false), которая увеличивает Errors. В ветви отказа вместо HitHazard проверьте selection.Errors >= 3 и установите Lost. Прогресс не растёт, неверный предмет не удаляется. Sequence.Errors аналогично считает ошибки вариантов 06–10.

5

Проверка

Проверьте запас при 1 и после следующего расхода: он равен 0, Lost, ввод остановлен. Значение не отрицательное. Для ошибок выполните ровно три неверных действия: правильный прогресс остаётся прежним. Повтор возвращает запас, ошибки и интерфейс к начальному состоянию.

ПМ.12 / Технические отличия вариантов

Потеря груза и возврат аптечки в варианте 14

1

Сохраните исходную точку

В GameManager добавьте private Vector2 cargoOrigin;. В Collect после принятого delivery.Pickup, перед Destroy(item.gameObject), присвойте cargoOrigin = item.transform.position;. Доставленные предметы больше не считаются переносимым грузом.

2

Добавьте восстановление в HitHazard

Сразу после проверки IsPlaying и invulnerableUntil, перед уменьшением hearts, добавьте: if (delivery.Drop()) { var dropped = CreateSpriteObject("Crystal", cargoOrigin, new Vector2(0.62f,0.82f), Color.yellow, Shape.Diamond,5); dropped.AddComponent<PolygonCollider2D>(); dropped.AddComponent<Collectible>(); createdObjects.Add(dropped); }. Drop снимает только флаг груза и не обнуляет уже доставленное количество.

3

Проверка сохранности

Возьмите аптечку, коснитесь опасности: груз пуст, аптечка появилась в исходной точке, прочность уменьшилась. Вход на площадку после потери не засчитывает доставку. Повторно подберите аптечку и доставьте: один зачёт. Новый запуск удаляет восстановленный предмет вместе с остальными и заново создаёт исходный набор.

ПМ.12 / Настройка собственного проекта

Цель варианта и достаточное число предметов

1

Измените Target и начальные данные

В GameManager найдите private const int Target = 5; и замените 5 числом из своей карточки. Для вариантов 16–20 это соответственно 6, 6, 8, 10 и 9. В crystalPositions добавьте столько разных доступных координат, сколько предметов требуется для выполнения. Не размещайте их внутри стен и опасностей. Для выборки добавьте отдельно неверные типы: они не входят в необходимое количество правильных предметов.

2

Уберите фиксированную подсказку

В RestartGame замените строку hint с числом 5 на интерполированную строку с Target и назначением своего показателя. В TryFinishLevel проверяйте соответствующее правило: число доставленных предметов, правильный состав комплекта или все открытые проходы. Подпись интерфейса должна отражать этот же показатель. Если карточка требует достижение выхода, одного выполненного счётчика недостаточно: проверка состояния выполняется при контакте с ExitZone.

3

Проверка достижимости

Проверьте границу Target - 1: победа ещё недоступна. Достигните Target без демонстрационных F6–F8 и выполните оставшееся условие карточки: доставка, полный комплект или выход. В вариантах с вместимостью число рейсов определяется ограничением: 16 — два, 17 — три, 18 — два, 19 — два, 20 — три при полной загрузке. Все необходимые предметы должны существовать на карте; лишнее количество в руках не заменяет доставку.

ПМ.12 / Уточнения самостоятельного правила

Состав комплекта и условие завершения

1

Точные требования по карточкам

Для 21 и 25 задайте тип 0 — трава/образец, тип 1 — кристалл/реагент, требования 2, 1, 0. Для 22–24 три типа имеют требования 1, 1, 1. В GameManager добавьте readonly int[] required = { 2, 1, 0 }; либо { 1, 1, 1 };. В Collect используйте kit.Take(item.itemType, required[item.itemType]); вместо фиксированной тройки условных значений. Перед обращением к массиву проверьте item.itemType < 0 || item.itemType >= required.Length и откажите этому предмету. В TryFinishLevel используйте kit.Complete(required[0], required[1], required[2]).

2

Немедленная победа или зона результата

Варианты 03, 05, 07–10 и 22 завершаются после своего последнего правильного действия: в конце успешной ветви Collect вызовите TryFinishLevel. Для 01, 02, 04, 06, 21, 23–25 оставьте отдельный контакт с местом завершения или доставки: в Collect победу не вызывайте. Для доставок и вместимости победа считается по Delivered, после разгрузки. Для 26–30 результат требует также достижение выхода/цели согласно карточке.

3

Проверка состава

В 21/25 два предмета типа 0 не заменяют реагент типа 1. Третий тип не обязателен и не должен блокировать финиш. В 22–24 повтор первого типа не заполняет второй и третий. Проверьте неполный состав, полный состав, лишний предмет и сброс каждого счётчика.

Ключ в замке, мосты и спасённый робот

1

Ворота варианта 26

Храните флаги ключей через Access и дополнительно bool gateAOpened, gateBOpened. В Collect ключ только вызывает access.Activate; ворота здесь не отключайте. Создайте KeyGate по образцу ExitZone, добавьте public int keyIndex; public Collider2D barrier;. Вместо TryFinishLevel вызовите GameManager.Instance.TryOpenGate(keyIndex, barrier);. В GameManager добавьте метод, который проверяет IsPlaying и access.Enabled(index), затем отключает barrier.enabled и ставит соответствующий gateOpened=true. Создайте перед стеной отдельную зону с BoxCollider2D isTrigger=true и KeyGate, задайте индекс 0/1 и ссылку на collider соответствующей стены. TryFinishLevel проверяет оба gateOpened и контакт с выходом, а RestartGame сбрасывает оба признака и включает барьеры.

2

Мосты варианта 29

В GameManager рядом с Access добавьте Sequence bridgeOrder. До access.Activate в Collect проверяйте bridgeOrder.Take(item.itemIndex, false); при отказе учитывайте ошибку/падение и return. Присвойте трём зарядам номера 1–3. Три барьера символизируют закрытые участки маршрута; после правильного заряда отключайте только барьер с номером itemIndex. В RestartGame вызывайте bridgeOrder.Reset и включайте все три барьера. Финиш: access.All(3) и контакт с порталом. Проверка: второй заряд первым не открывает участок; первый разрешает только первый участок; повтор не заменяет следующий.

3

Робот варианта 30

Считайте управляемый Player роботом, а экранные кнопки — пультом оператора; явно подпишите это в интерфейсе. Два разных рубильника устанавливают Access-флаги и снимают collider двери при All(2). Само открытие двери не вызывает победу. ExitZone принимает физический контакт именно PlayerController робота, а TryFinishLevel проверяет All(2). Повтор рубильника учитывайте как ошибку системы, не как второй флаг; при третьей ошибке Lost. В RestartGame сбрасывайте Access, ошибки, дверь и позицию робота. Проверьте повтор первого рубильника, второй рубильник, открытие двери без победы, затем реальное движение робота до выхода.

4

Повтор рубильника в варианте 30

Рубильник должен остаться на сцене после активации, иначе повторить контакт с ним невозможно. Для этого в ветви варианта 30 не выполняйте Remove и Destroy. После принятой access.Activate обновите состояние двери и завершите Collect через return. В отказе повторной Activate увеличьте отдельный счётчик системных ошибок и завершите обработчик. Проверяйте новый вход после выхода из триггера, а не каждый кадр пребывания внутри. При RestartGame восстановите ошибки и Access; рубильники остаются доступными.

ПМ.12 / Создание, объяснение и проверка

Перенос правила на свою предметную область

1

Сопоставьте сущности

Составьте соответствие: Crystal → предмет области; Hazard → её опасность; Exit → место завершения или разгрузки. Запишите номер/тип, изменение состояния, отказ и условие завершения. У каждого предмета должно быть понятное действие, а у каждого состояния — место сброса.

2

Различия внутри группы

Не все карточки требуют одинакового штрафа. Уточните сброс последовательности, повреждение, замедление, вместимость, количество доставок, состав набора и связи двери с ключом. Эти различия реализуются в обработчиках выбранной модели, а не новой технологией. Для вариантных ошибок ведите отдельный errors и проверяйте его предел; не называйте потерю жизни ошибкой, если карточка задаёт другое правило.

3

Изменяйте по одному

Сначала замените данные и подписи; проверьте исходную игру. Затем измените принятие контакта, затем интерфейс и условие финиша, затем полный сброс. После каждого законченного изменения сохраните, скомпилируйте и выполните конкретную проверку. Не переносите сразу все шесть механик.

4

Скачать форму «Отчет о выполнении.docx»

Скачать форму «Отчет о выполнении.docx»

Запишите исходные условия, изменения, правильный и неправильный сценарии, результаты дополнительных требований и вывод. Не подставляйте результаты демонстрационного примера вместо собственных.

Дополнительно: откуда брать изображения

1

Необязательное оформление

Этот раздел выполняйте только по желанию после работающего индивидуального правила. Обязательная игра использует формы, создаваемые кодом. Для её выполнения не нужны скачивание графики и поиск внешних материалов.

2

1. Самый безопасный источник

Используйте собственные рисунки, изображения преподавателя или файлы из выданного набора. Для простого объекта достаточно PNG размером 64×64, 128×128 или 256×256 пикселей с прозрачным фоном.

3

Открыть Kenney Assets для выбора графики

Открыть Kenney Assets для выбора графики

Можно использовать Kenney Assets: https://kenney.nl/assets. На официальной странице поддержки указано, что материалы на страницах наборов распространяются как CC0. Скачивайте ZIP со страницы выбранного набора и сохраняйте приложенный файл лицензии.

4

Открыть OpenGameArt для выбора графики и проверки лицензии

Открыть OpenGameArt для выбора графики и проверки лицензии

Допускается https://opengameart.org, но у каждого файла своя лицензия. Перед скачиванием откройте блок License, запишите автора, ссылку и условия указания авторства. Если лицензия непонятна, не используйте файл.

5

4. Что нельзя делать

Не копируйте случайную картинку из поиска Google или чужой игры. Не используйте файл без сведений о лицензии. В README добавьте раздел «Графические материалы»: название файла, автор, источник и лицензия.

Этап 2 / До переноса в Unity

Дополнительно: как подготовить скачанный файл

1

5. Распакуйте архив

Откройте папку Downloads или «Загрузки». Дважды щёлкните скачанный ZIP. Откройте распакованную папку и найдите каталог PNG, Sprites или отдельно нужный .png-файл.

2

6. Выберите подходящее изображение

Для игрока выберите один вид сверху или простой значок. Для предмета и опасности используйте разные силуэты. Прозрачный фон отображается в графическом редакторе клетчатым полем. JPG для объектов с прозрачностью не подходит.

3

7. Переименуйте файл

Используйте короткое имя латинскими буквами без пробелов: player.png, crystal.png, hazard.png, exit.png, wall.png. Не оставляйте имена вида image (1).png — их легко перепутать.

4

8. Не переносите весь архив

В Unity копируйте только выбранные PNG и файл лицензии. Не помещайте в Assets исходный ZIP, рекламные изображения, PSD-файлы и ненужные варианты — они увеличивают проект и замедляют импорт.

Этап 2 / Project → Assets → Art → Sprites

Дополнительно: как загрузить картинку в Unity

1

9. Создайте папки

В Project нажмите Assets. Щёлкните правой кнопкой → Create → Folder, введите Art и нажмите Enter. Дважды щёлкните Art, снова создайте Folder с именем Sprites и откройте её. Вверху Project должен быть путь Assets > Art > Sprites.

2

10. Способ через перетаскивание

Откройте Finder на macOS или Проводник на Windows. Уменьшите окно так, чтобы одновременно видеть файл и Unity. Перетащите player.png в свободное место окна Project внутри Assets/Art/Sprites. Отпустите кнопку мыши и дождитесь импорта.

3

11. Способ через меню

Если перетаскивание неудобно, в Unity выберите Assets → Import New Asset. В открывшемся окне найдите PNG, нажмите его один раз, затем нажмите Import. Файл появится в текущей папке Project.

4

12. Проверьте результат

В Project появилась миниатюра с именем файла. Если вместо неё виден белый документ или Unity показала красную ошибку, проверьте расширение .png и откройте Console. Не переименовывайте импортированный файл вне Unity после его использования.

Этап 2 / Inspector → Apply

Дополнительно: настройки картинки

1

13. Выделите PNG

Один раз нажмите player.png в Project. Справа откроется Texture Import Settings. Если Inspector пуст, выберите Window → General → Inspector и снова нажмите файл.

2

14. Установите тип Sprite

В поле Texture Type выберите Sprite (2D and UI). В Sprite Mode выберите Single. Pixels Per Unit установите 100. Mesh Type выберите Full Rect. Включите Alpha Is Transparency, если у PNG прозрачный фон.

3

15. Выберите фильтрацию

Для обычной гладкой графики выберите Filter Mode: Bilinear. Для пиксельной графики выберите Point (no filter), иначе края станут размытыми. Compression для учебного проекта установите None.

4

16. Нажмите Apply

Внизу Inspector нажмите Apply. Не переходите к другому файлу до нажатия. Повторите пункты 13–16 для crystal.png, hazard.png, exit.png и wall.png. Эти параметры соответствуют рекомендациям Unity для Sprite (2D and UI).

ПМ.12 / Создание, объяснение и проверка

Дополнительно: как подключить PNG к автоматически созданному объекту

1

Папка и загрузка

Импортируйте нужный спрайт, затем в Assets создайте Resources и перенесите в него player.png. Resources.Load<Sprite>("player") ищет имя без расширения внутри Resources. Это простой вариант для учебной игры; исходное изображение должно иметь Texture Type Sprite (2D and UI).

2

Замените назначение renderer.sprite в CreateSpriteObject

```
Sprite picture = name == "Player" ? Resources.Load<Sprite>("player") : null;
renderer.sprite = picture != null ? picture : CreateSprite(shape, color);
```

3

Место и проверка

Assets/Scripts/GameManager.cs, CreateSpriteObject: замените только renderer.sprite = CreateSprite(shape, color);. Сохраните и запустите Game. Изменяется игрок, остальные фигуры сохраняются. Если картинка не загрузилась, геометрический спрайт остаётся: проверьте Resources, имя и тип импорта. Не добавляйте ручного Player в Hierarchy рядом с автоматически созданным.

ПМ.12 / Создание, объяснение и проверка

Подтвердите результат и передайте проект пользователю

1

Проверка другой копии

Остановите Play, File → Save, закройте Unity. Создайте контрольную копию Assets, Packages и ProjectSettings в отдельной папке. Откройте её через Unity Hub → Projects → Add → Add project from disk, выберите корень с этими тремя папками. Это проверка переносимости: не требуется переносить Library.

2

Сценарии после переноса

Откройте Main; Console без красных ошибок. Повторите начало, управление, индивидуальное правило, ранний выход, победу, поражение и повтор. Запишите наблюдения, версию Unity и дату в отчёте. Если другой экран обрезает поле, проверьте Game 16:9 и параметры камеры.

3

Репозиторий проекта

При сдаче проекта приложите ссылку на его репозиторий. В README своего проекта объясните запуск Main, управление, правило и известные ограничения. Не подставляйте репозиторий преподавателя. Состав загружаемых материалов определяется ЛМС; здесь он не дублируется.

4

Мобильная проверка

Обязательный результат этого комплекта — прототип в Game View с экранным управлением. Сборка Android дополнительная: если она выполняется, отдельно проверьте удержание экранных кнопок, остановку после отпускания, ориентацию и читаемость на устройстве. Успех в редакторе не подтверждает работу на телефоне.